

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Кафедра «Информационной безопасности систем и технологий»

Курсовая работа
по дисциплине «Сетевые технологии» на тему
«Разработка приложений с сетевым взаимодействием»
ПГУ.100502.С1.О.32.КР.22ПР19.01.ПЗ

Специальность – 10.05.02 Информационная безопасность
телекоммуникационных систем

Специализация – 7 Разработка защищённых телекоммуникационных
систем

Выполнил студент: _____ Мочалин А.А.

Группа: 22ПТ1

Руководитель: старший преподаватель
Щербакова А.Ю.

Работа защищена с оценкой: _____

Дата защиты: _____

Пенза 2026

УТВЕРЖДАЮ

Зав. кафедрой ИБСТ

к.т.н., доцент

С.Л.Зефиров

13 03 2025 г.

ЗАДАНИЕ

на курсовую работу

по теме: Разработка приложений с сетевым взаимодействием.

1 Дисциплина: Сетевые технологии.

2 Вариант задания: 7.

3 Студент: Мочалин А.А.

4 Группа: 22ПТ1.

5 Цель: разработка сетевого приложения с архитектурой клиент-сервер.

6 Исходные данные на проектирование:

— требования к сетевому взаимодействию:

— используемый протокол транспортного уровня – TCP;

— в протоколе прикладного уровня должны быть предусмотрены возможности обмена информацией о возникающих ошибках при сетевом взаимодействии;

— требования к программе-клиенту:

— программа-клиент должна получать с сервера и сохранять на локальном компьютере указанный файл;

— в программе-клиенте должна быть предусмотрена обработка ошибок, возникающих при сетевом взаимодействии;

— требования к программе-серверу:

— программа-сервер должна передавать запрошенный программой-клиентом файл;

— в программе-сервере должен быть реализован механизм протоколирования длительности передачи файлов каждой программе-клиенту;

— в программе-сервере должно быть реализовано разделение полномочий пользователей программ-клиентов: пользователь с правами администратора сервера должен иметь возможность просмотра, редактирования и очистки журнала длительности передачи файлов;

— в программе-сервере должна быть предусмотрена обработка ошибок, возникающих при сетевом взаимодействии;

— максимальное количество одновременно подключенных программ-клиентов – 3;

— при попытке подключения программы-клиента сверх максимального количества, ей должно возвращаться сообщение об ошибке подключения;

— должна быть реализована аутентификация пользователя программы-клиента при подключении к серверу;

— должен быть разработан протокол прикладного уровня, предназначенный для реализации сетевого взаимодействия между программой-клиентом и программой-сервером;

— должна быть проведена проверка работоспособности программ;

— должно быть разработано руководство пользователя.

7 Структура проекта:

7.1 Пояснительная записка (содержание работы):

— разработка протокола прикладного уровня;

— программная реализация клиента и сервера;

— проверка работоспособности разработанных программ;

— разработка руководства пользователя.

7.2 Графическая часть:

UML-диаграммы протокола прикладного уровня.

7.3 Экспериментальная часть:

Тестирование разработанных программ.

8 Календарный план выполнения работы:

— оформление задания	к 17.02.2025;
— разработка протокола прикладного уровня	к 11.03.2025;
— программная реализация клиента и сервера	к 14.04.2025;
— проверка работоспособности программ	к 05.05.2025;
— разработка руководства пользователя	к 12.05.2025;
— оформление отчета	к 22.05.2025.

Задание получил « 12 » февраля 2025 г.

Студент

Руководитель



А.А. Мочалин

А.Ю. Щербакова

Оглавление

Введение.....	5
1 Разработка протокола прикладного уровня.....	6
1.1 Функциональные возможности программного комплекса.....	6
1.2 Диаграмма состояний программы-клиента.....	8
1.3 Диаграмма состояний программы-сервера.....	9
1.4 Типы запросов и ответов протокола.....	10
1.5 Диаграммы последовательности взаимодействия.....	11
1.6 Форматы сообщений протокола и описание полей.....	14
2 Разработка клиента и сервера.....	22
3 Тестирование разработанных программ.....	28
4 Разработка руководства пользователя.....	31
Заключение.....	33
Список используемых источников.....	34
Приложение А Диаграммы состояний и последовательности.....	35
Приложение Б Код программы-клиента.....	42
Приложение В Код программы-сервера.....	80

Введение

В современных распределенных информационных системах задача оперативного и защищенного обмена файлами между узлами сети является одной из приоритетных. На текущий момент существует множество готовых решений, однако они зачастую ориентированы на облачные инфраструктуры или не предоставляют администратору механизмов детального низкоуровневого мониторинга трафика в реальном времени. Основной трудностью остается обеспечение высокой скорости передачи данных при сохранении прозрачности процесса аудита и гибкого управления квотами подключений.

Основанием для выполнения данной работы послужила необходимость разработки специализированного клиент-серверного приложения на базе современных веб-технологий, способного осуществлять передачу данных по протоколу TCP с использованием технологии WebSocket. Исходными данными для проектирования стали требования к реализации двустороннего обмена и эффективной передачи файлов по частям (сегментами) с визуализацией прогресса.

Актуальность темы подтверждается возрастающими требованиями к безопасности и управляемости сетевых служб в рамках импортозамещения и перехода на отечественные стандарты разработки ПО. Интеграция панели администратора с возможностью просмотра детальных логов и статистики транзакций отвечает современным условиям эксплуатации систем в критически важных сегментах сети.

1 Разработка протокола прикладного уровня

1.1 Функциональные возможности программного комплекса

Разрабатываемый программный комплекс представляет собой систему, предназначенную для автоматизации процессов обмена данными между узлами распределенной сети. Проектируемая функциональность системы направлена на обеспечение контролируемой передачи файлов с возможностью мониторинга безопасности и аудита действий пользователей в реальном времени.

В рамках реализации проекта предусматривается внедрение следующих ключевых функций:

- аутентификация и разграничение прав доступа;
- многопоточная загрузка файлов;
- поточное скачивание данных с сервера;
- мониторинг сетевой активности и протоколирование;
- защита от перегрузок.

Одной из ключевых задач разработки является проектирование механизмов аутентификации и разграничения прав доступа. Требуется реализовать надежный механизм проверки подлинности пользователей с применением защищенных токенов для обеспечения безопасности сетевого взаимодействия. Данное решение призвано обеспечить строгое разделение полномочий субъектов доступа: роль «Администратор» должна получить доступ к инструментам расширенного мониторинга сетевой активности и системным логам, в то время как функциональные возможности роли «Пользователь» планируется ограничить исключительно операциями загрузки и скачивания файлов.

Для повышения эффективности использования пропускной способности сети и исключения блокировок основного цикла событий сервера требуется

реализовать механизм сегментированной передачи данных. Планируется обеспечить загрузку файлов путем разбиения входного бинарного потока на пакеты фиксированного размера. С целью оптимизации дисковых операций и поддержания высокой отзывчивости интерфейса администратора, необходимо вынести процедуры записи данных на накопитель в изолированные вычислительные потоки. Данная архитектура призвана гарантировать стабильную работу системы при одновременной обработке нескольких объектов большого объема и предотвращать снижение производительности сервера в условиях высокой сетевой нагрузки.

Для минимизации нагрузки на вычислительные сервера при передаче данных пользователю требуется реализовать механизм потокового скачивания. Необходимо обеспечить последовательное чтение данных с накопителя и их порционную отправку в сеть. Данный подход призван исключить необходимость предварительной загрузки всего файла в оперативную память сервера, что должно гарантировать стабильную работу серверной части приложения при одновременной обработке запросов на получение объектов большого объема.

Для обеспечения прозрачности верификации корректности сетевого взаимодействия необходимо реализовать механизмы мониторинга сетевой активности и протоколирования. Требуется обеспечить непрерывный аудит событий безопасности, включая автоматическую фиксацию моментов подключения и отключения пользователей, а также протоколирование возникающих системных ошибок. В рамках разрабатываемого протокола необходимо организовать запись параметров каждого передаваемого пакета в базу данных с сохранением информации о его типе, размере и направлении. Данный подход призван сформировать детальный журнал сетевой активности для последующего анализа безопасности и оперативного контроля состояния системы администратором.

В логику сервера необходимо интегрировать механизм контроля

соединений, ограничивающий число одновременных активных сессий до трех, что служит базовой защитой от атак, направленных на отказ в обслуживании.

1.2 Диаграмма состояний программы-клиента

Для визуализации логики работы и межуровневого взаимодействия компонентов системы была разработана диаграмма состояний программы-клиента. Данная диаграмма описывает жизненный цикл функционирования клиентского приложения и логику переходов между различными режимами работы в зависимости от пользовательского ввода и сигналов, поступающих.

Диаграмма состояний программы-клиента представлена в приложении А на рисунке А.1.

После запуска программа-клиент переходит в состояние «Подключение к серверу», в котором открывается соединение и отправляется токен аутентификации. При получении от сервера сообщения об успешном подключении программа переходит в состояние «Ожидание команды». В случае ошибки или истечения срока действия токена программа возвращается в состояние «Подключение к серверу».

Из состояния «Ожидание команды» возможны три перехода. При выборе пользователем файла для загрузки программа переходит в состояние «Передача файла», при выборе файла для скачивания – в состояние «Получение файла». При отправке команды закрытия соединения программа переходит в конечное состояние и завершает работу.

В состоянии «Передача файла» программа отправляет серверу заголовок файла, затем данные файла, разделённые на равные сегменты и сигнал завершения передачи. При успешной передаче программа возвращается в состояние «Ожидание команды». При ошибке – переходит в состояние «Ошибка передачи», откуда после действия пользователя возвращается обратно в «Ожидание команды». Если файл не был выбран, переход также

осуществляется обратно в «Ожидание команды».

В состоянии «Получение файла» программа отправляет запрос на скачивание файла, принимает метаданные, накапливает сегменты данных и сохраняет файл по завершении передачи. При успешном скачивании программа возвращается в состояние «Ожидание команды». При ошибке – переходит в состояние «Ошибка скачивания», откуда после действия пользователя возвращается в «Ожидание команды». Если файл не был выбран, переход также осуществляется обратно в «Ожидание команды».

1.3 Диаграмма состояний программы-сервера

Для описания логики функционирования серверной части системы была разработана диаграмма состояний программы-сервера. Данная диаграмма отражает жизненный цикл серверного приложения и определяет последовательность переходов между режимами работы в зависимости от поступающих запросов и результатов их обработки.

Диаграмма состояний программы-сервера представлена в приложении А на рисунке А.2.

После запуска программа-сервер переходит в состояние «Инициализация», в котором выполняется запуск HTTP-сервера, регистрация обработчиков запросов и создание вспомогательного потока для файловых операций. По завершении инициализации программа переходит в состояние «Ожидание подключения».

В состоянии «Ожидание подключения» сервер принимает входящие соединения и осуществляет периодическую проверку активности установленных соединений. При поступлении запроса на подключение программа переходит в состояние «Обработка запроса на подключение».

В состоянии «Обработка запроса на подключение» сервер проверяет допустимость нового соединения: контролируется соответствие числа активных

клиентов установленному ограничению, а также выполняется верификация переданного токена аутентификации. Если запрос признан корректным, сервер отправляет подтверждение и переходит в состояние «Ожидание запроса на передачу». В противном случае отправляется сообщение об ошибке и сервер возвращается в состояние «Ожидание подключения».

Из состояния «Ожидание запроса на передачу» возможны три перехода. При получении запроса на загрузку файла программа переходит в состояние «Приём и запись файла», при получении запроса на скачивание – в состояние «Отправка файла клиенту». При получении сигнала закрытия соединения выполняется запись события в журнал и программа возвращается в состояние «Ожидание подключения».

В состоянии «Приём и запись файла» сервер принимает поступающие данные, передаёт их вспомогательному потоку для записи на диск и фиксирует показатели скорости передачи в базе данных. По завершении приёма файла клиенту направляется подтверждение, всем подключённым клиентам рассылается уведомление об обновлении списка файлов, и сервер возвращается в состояние «Ожидание запроса на передачу».

В состоянии «Отправка файла клиенту» сервер проверяет наличие запрошенного файла на диске, после чего осуществляет его потоковую передачу клиенту равными блоками. По завершении передачи клиенту направляется сигнал окончания и сервер возвращается в состояние «Ожидание запроса на передачу». Если запрошенный файл не найден, клиенту отправляется сообщение об ошибке и сервер также возвращается в состояние «Ожидание запроса на передачу».

1.4 Типы запросов и ответов протокола

На основе разработанных диаграмм состояний программы-клиента и программы-сервера были определены необходимые типы сообщений протокола

прикладного уровня. Каждое сообщение соответствует событию, вызывающему смену состояния одной из взаимодействующих сторон. Сообщения протокола построены по схеме запрос-ответ: клиент инициирует взаимодействие, направляя запрос серверу, сервер обрабатывает запрос и возвращает ответ, сигнализирующий об успешности или неудаче выполненной операции.

Протокол реализован поверх транспортного протокола WebSocket. Сообщения-запросы передаются в бинарном формате, сообщения-ответы – в текстовом формате JSON. Перечень запросов и соответствующих им ответов представлен в таблице 1.

Таблица 1 – Запросы и ответы протокола

Запрос клиент → сервер	Ответ сервер → клиент
Запрос на подключение (AUTH)	Успешное подключение (auth_success)
	Ошибка подключения (error)
Начало передачи файла (FILE_START)	–
Передача данных файла (FILE_CHUNK)	–
Завершение передачи файла (FILE_END)	Файл принят (file_success)
	Ошибка сохранения (error)
Запрос на скачивание (DOWNLOAD_REQUEST)	Начало отдачи файла (DOWNLOAD_START)
	Файл не найден (error)
–	Передача данных файла (DOWNLOAD_CHUNK)
–	Завершение отдачи файла (DOWNLOAD_END)
Закрытие соединения (DISCONNECT)	–

Помимо ответов на запросы, сервер формирует широковещательное сообщение `file_list_update`, которое направляется всем подключённым клиентам при успешном завершении загрузки нового файла. Данное сообщение не является ответом на конкретный запрос и инициируется сервером самостоятельно.

1.5 Диаграммы последовательности взаимодействия

Для описания порядка обмена сообщениями между программой-клиентом и программой-сервером были разработаны диаграммы последовательности. Каждая диаграмма соответствует одному из возможных сценариев взаимодействия. Все сообщения, отображённые на диаграммах, соответствуют типам запросов и ответов, определённым в таблице 1.

Диаграмма последовательности успешной загрузки файла представлена в приложении А на рисунке А.3.

Взаимодействие инициируется программой-клиентом, которая направляет серверу запрос на авторизацию. Сервер проверяет переданные данные и при их корректности возвращает подтверждение успешной авторизации. После установления авторизованного соединения клиент приступает к передаче файла, отправляя запрос на передачу файла. Со стороны программы-клиента последовательно отправляются метаданные файла, содержимое файла и сигнал окончания передачи. Программа-сервер принимает данные, сохраняет файл и направляет серверу подтверждение успешной отправки. Помимо этого, сервер рассылает всем подключённым клиентам уведомление об обновлении списка файлов. По завершении работы клиент направляет серверу сигнал завершения сессии.

Диаграмма последовательности успешного скачивания файла представлена в приложении А на рисунке А.4.

Взаимодействие инициируется программой-клиентом, которая направляет серверу запрос на авторизацию. Сервер проверяет переданные данные и при их корректности возвращает подтверждение успешной авторизации. После установления авторизованного соединения клиент направляет серверу запрос на скачивание файла. Сервер проверяет наличие запрошенного файла и при его наличии последовательно передаёт клиенту метаданные файла, содержимое и сигнал окончания скачивания файла. По завершении передачи клиент направляет серверу подтверждение успешного скачивания. По завершении работы клиент направляет серверу сигнал

завершения сессии.

Диаграмма последовательности при ошибке авторизации представлена в приложении А на рисунке А.5.

Взаимодействие инициируется программой-клиентом, которая направляет серверу запрос на авторизацию. Сервер выполняет проверку переданных данных и при обнаружении их недействительности или некорректности возвращает клиенту сообщение об ошибке с указанием причины. После этого сервер принудительно закрывает соединение.

Диаграмма последовательности при превышении лимита подключений представлена в приложении А на рисунке А.6.

Взаимодействие инициируется программой-клиентом, которая направляет серверу запрос на авторизацию. Сервер проверяет текущее число активных подключений и при обнаружении превышения допустимого предела возвращает клиенту сообщение об ошибке с указанием причины. После этого сервер принудительно закрывает соединение.

Диаграмма последовательности при запросе несуществующего файла представлена в приложении А на рисунке А.7.

Взаимодействие инициируется программой-клиентом, которая направляет серверу запрос на авторизацию. Сервер проверяет переданные данные и при их корректности возвращает подтверждение успешной авторизации. После установления авторизованного соединения клиент направляет серверу запрос на скачивание файла. Сервер выполняет поиск запрошенного файла и при его отсутствии возвращает клиенту сообщение об ошибке с указанием причины. После этого сервер принудительно закрывает соединение.

Диаграмма последовательности при разрыве соединения по истечению времени ожидания представлена в приложении А на рисунке А.8.

После установления соединения сервер в соответствии с механизмом контроля активности периодически направляет клиенту запрос проверки

соединения. При отсутствии ответа со стороны клиента сервер выполняет повторную проверку. Если ответ повторно не получен, сервер расценивает соединение как неактивное и выполняет его принудительное закрытие.

Диаграмма последовательности при получении запроса без аутентификации представлена в приложении А на рисунке А.9.

Программа-клиент направляет серверу команду без предварительной отправки запроса на авторизацию. Сервер обнаруживает отсутствие установленной авторизованной сессии и немедленно выполняет принудительное закрытие соединения без направления каких-либо сообщений клиенту.

1.6 Форматы сообщений протокола и описание полей

Форматы сообщений протокола прикладного уровня определяют структуру данных, передаваемых между программой-клиентом и программой-сервером. Каждое сообщение содержит заголовок и тело. Заголовок представляет собой однобайтовый идентификатор типа сообщения. Тело содержит данные, необходимые для выполнения соответствующей операции, и может отсутствовать. Сообщения-запросы передаются в бинарном формате. Управляющие сообщения-ответы сервера передаются в текстовом формате JSON и содержат обязательное поле `type`, определяющее тип ответа.

Запрос на авторизацию (AUTH)

Направляется клиентом серверу сразу после установления соединения. Формат сообщения приведён на рисунке 1.

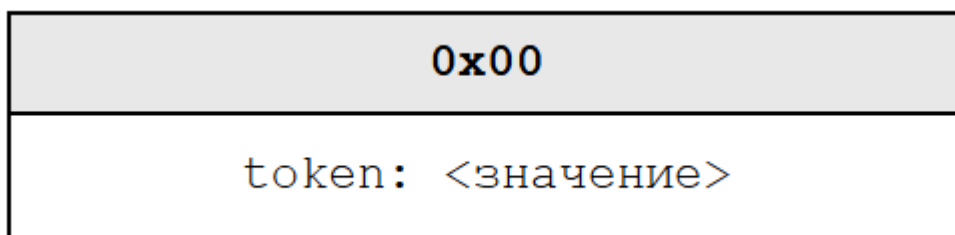


Рисунок 1 – Формат сообщения AUTH

Значения полей представлены в таблице 2.

Таблица 2 – Описание полей сообщения AUTH

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения AUTH, фиксированное значение 0x00
token	UTF-8 строка	JWT-токен, полученный при аутентификации. Используется сервером для идентификации пользователя.

Начало загрузки файла (FILE_START)

Направляется клиентом серверу перед началом передачи данных файла.

Формат сообщения FILE_START представлен на рисунке 2.

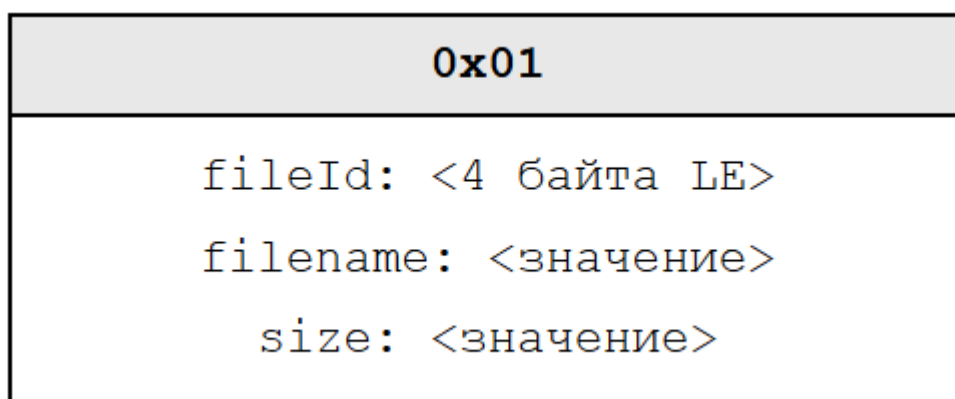


Рисунок 2 – Формат сообщения FILE_START

Сообщение состоит из трёх полей: однобайтового типа, четырёхбайтового идентификатора сессии передачи и JSON-объекта, содержащего метаданные файла (имя файла, размер файла, тип содержимого и т.д.). Значения полей представлены в таблице 3.

Таблица 3 – Формат сообщения FILE_START

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения FILE_START, фиксированное значение 0x01
fileId	uint32 LE, байты 1–4	Идентификатор сессии передачи файла. Случайное число, используемое для сопоставления последующих сообщений FILE_CHUNK и FILE_END с данной передачей
filename	UTF-8 строка, JSON	Исходное имя передаваемого файла
size	uint64, JSON	Общий размер файла в байтах. Используется сервером для расчёта прогресса приёма данных

Передача порции данных файла (FILE_CHUNK)

Направляется клиентом серверу в процессе трансляции содержимого файла. Сообщение передается многократно до полной отправки всех сегментов данных. Формат сообщения `FILE_CHUNK` представлен на рисунке 3.

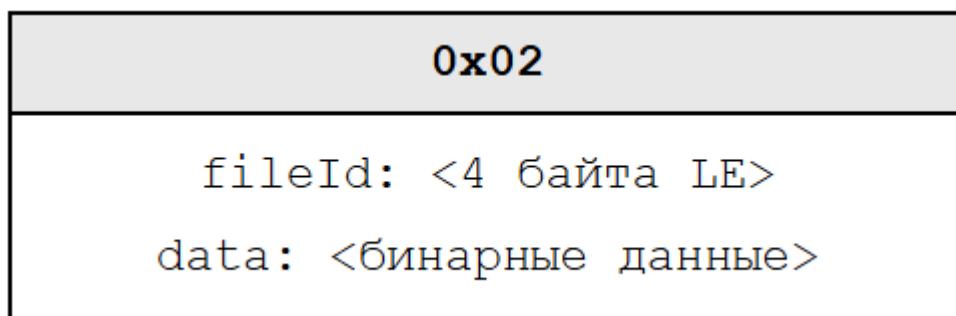


Рисунок 3 – Формат сообщения `FILE_CHUNK`

Сообщение состоит из трёх функциональных полей: однобайтового префикса типа, четырёхбайтового идентификатора сессии передачи и массива бинарных данных, представляющего собой фрагмент содержимого файла. Значения полей представлены в таблице 4.

Таблица 4 – Формат сообщения `FILE_CHUNK`

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения <code>FILE_CHUNK</code> .
fileId	uint32 LE, байты 1–4	Идентификатор файла из <code>FILE_START</code> . Должен совпадать с ранее переданным значением.
data	bytes, байты 5–N	Бинарные данные фрагмента файла произвольного размера. Сервер передаёт данные в <code>Worker Thread</code> для записи на диск.

Запрос на скачивание файла (`DOWNLOAD_REQUEST`)

Направляется клиентом серверу для инициирования передачи файла из хранилища на локальное устройство пользователя. Формат сообщения `DOWNLOAD_REQUEST` представлен на рисунке 4.

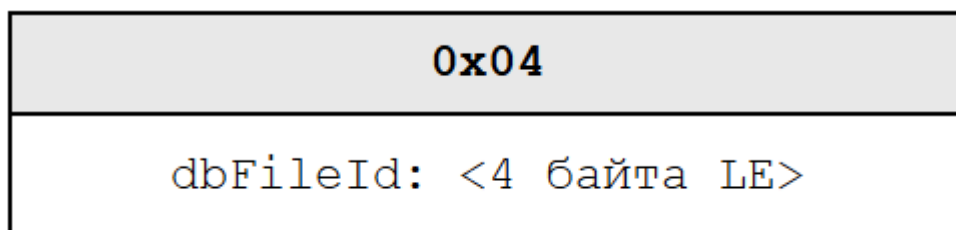


Рисунок 4 – Формат сообщения `DOWNLOAD_REQUEST`

Сообщение имеет фиксированную длину и состоит из двух полей: однобайтового префикса типа и четырёхбайтового идентификатора файла в базе данных. Значения полей представлены в таблице 5.

Таблица 5 – Формат сообщения **DOWNLOAD_REQUEST**

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения DOWNLOAD_REQUEST .
dbFileId	uint32 LE, байты 1–4	Первичный ключ записи в таблице transfers базы данных. Клиент получает это значение из ответа GET /api/files. Сервер использует его для поиска файла на диске.

Начало отдачи файла сервером (**DOWNLOAD_START**)

Направляется сервером клиенту в ответ на запрос скачивания. Данное сообщение служит для передачи атрибутов файла, необходимых для подготовки файловой системы на стороне клиента к приему данных. Формат сообщения **DOWNLOAD_START** представлен на рисунке 5.

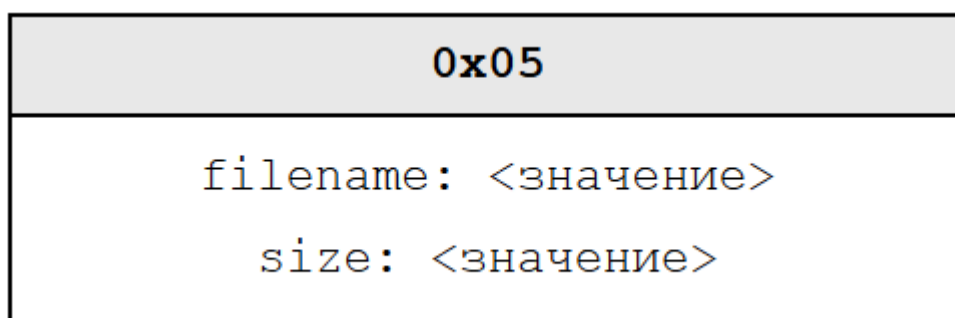


Рисунок 5 – Формат сообщения **DOWNLOAD_START**

Сообщение включает в себя два структурных компонента: однобайтовый префикс типа и объект метаданных в формате JSON. Описание полей сообщения приведено в таблице 6.

Таблица 6 – Формат сообщения **DOWNLOAD_START**

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения DOWNLOAD_START .
filename, size	UTF-8 JSON, байты 1–N	JSON-объект {"filename":"...", "size":...}. Клиент использует filename для сохранения файла, size – для расчёта прогресса скачивания.

Передача фрагмента данных при скачивании (DOWNLOAD_CHUNK)

Направляется сервером клиенту в процессе трансляции содержимого запрашиваемого файла. Сообщение передается циклически до полной передачи всего объема данных. Формат сообщения DOWNLOAD_CHUNK представлен на рисунке 6.

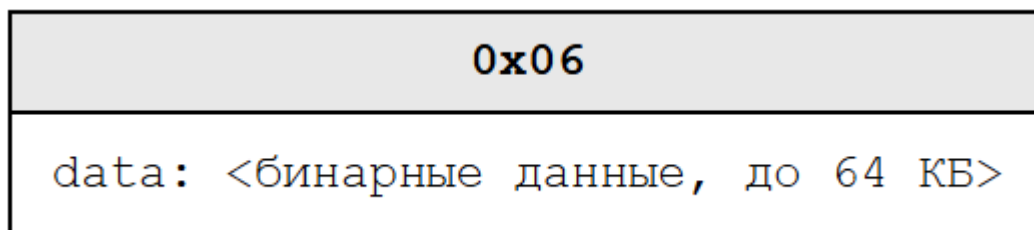


Рисунок 6 – Формат сообщения DOWNLOAD_CHUNK

Сообщение состоит из двух полей: однобайтового префикса типа и массива бинарных данных. Описание полей сообщения приведено в таблице 7.

Таблица 7 – Формат сообщения DOWNLOAD_CHUNK

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения DOWNLOAD_CHUNK.
data	bytes, байты 1–N	Фрагмент бинарных данных файла. Размер блока: до 65 536 байт (highwaterMark 64 КБ). Клиент накапливает сегменты в массиве до получения DOWNLOAD_END.

Завершение передачи файла (DOWNLOAD_END)

Направляется сервером клиенту для сигнализации об успешном окончании трансляции данных. Данное сообщение является маркером конца потока и не содержит дополнительной полезной нагрузки. Формат сообщения DOWNLOAD_END представлен на рисунке 7.

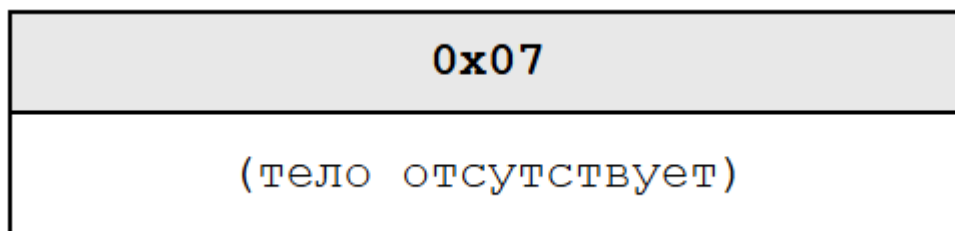


Рисунок 7 – Формат сообщения DOWNLOAD_END

Сообщение имеет фиксированный размер и состоит только из префикса

типа. После получения клиент собирает файл из накопленных сегментов и инициирует его сохранение на диск. Описание поля приведено в таблице 8.

Таблица 8 – Формат сообщения **DOWNLOAD_END**

Поле	Тип	Описание
opcode	uint8, 1 байт	Тип сообщения DOWNLOAD_END .

Подтверждение аутентификации (**auth_success**)

Направляется сервером клиенту после успешного прохождения проверки прав доступа. Данное сообщение переводит клиентское приложение в режим готовности к файловым операциям. Формат сообщения представлен на рисунке 8.

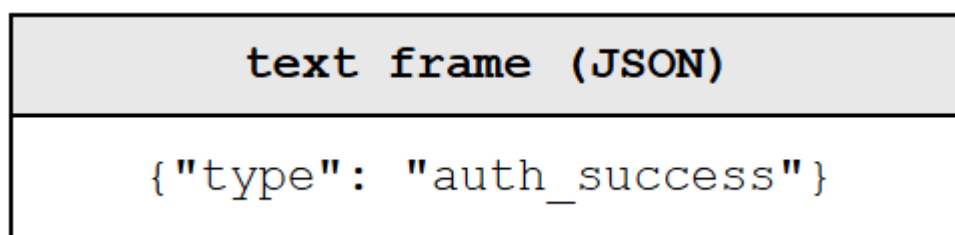


Рисунок 8 – Формат управляющего сообщения **auth_success**

В отличие от бинарных запросов, подтверждение передается в виде текстового сообщения. Структура сообщения представляет собой объект **JSON**, описание полей которого приведено в таблице 9.

Таблица 9 – Формат сообщения **auth_success**

Поле	Тип	Описание
type	uint8, 1 байт	Фиксированное значение "auth_success". Означает успешную верификацию JWT и начало авторизованной WebSocket-сессии.

Подтверждение успешного приема файла (**file_success**)

Направляется сервером клиенту после окончательной записи всех фрагментов данных на диск. Данное сообщение служит уведомлением о том, что операция загрузки завершена корректно. Формат сообщения представлен на рисунке 9.

Сообщение передается в виде текстового сегмента в формате JSON. Описание полей объекта приведено в таблице 10.

text frame (JSON)
<pre>{ "type": "file_success", "fileId": <значение> }</pre>

Рисунок 9 – Формат управляющего сообщения file_success

Таблица 10 – Формат сообщения file_success

Поле	Тип	Описание
type	строка UTF-8	Фиксированное значение "file_success".
fileId	number	Идентификатор файловой сессии из FILE_START. Подтверждает, что конкретный файл был успешно принят и сохранён.

Оповещение об обновлении хранилища

Направляется сервером всем подключенным клиентам в режиме широковещательной рассылки сразу после успешного завершения операции по приему нового файла. Данный механизм обеспечивает актуальность отображаемых данных в интерфейсе без необходимости ручной перезагрузки страницы пользователем. Формат сообщения представлен на рисунке 10.

text frame (JSON)
<pre>{ "type": "file_list_update" }</pre>

Рисунок 10 – Формат управляющего сообщения file_list_update

Сообщение передается в виде текстового фрейма. Структура объекта JSON приведена в таблице 11.

Таблица 11 – Формат сообщения `file_list_update`

Поле	Тип	Описание
<code>type</code>	строка UTF-8	Фиксированное значение <code>"file_list_update"</code> . Служит сигналом о необходимости синхронизации данных.

Сообщение об ошибке (`error`)

Направляется сервером клиенту в случае возникновения исключительных ситуаций, нарушения протокола или ошибок доступа к ресурсам. Данное сообщение позволяет клиентскому приложению корректно обработать инцидент и проинформировать пользователя о причинах сбоя. Формат сообщения представлен на рисунке 11.

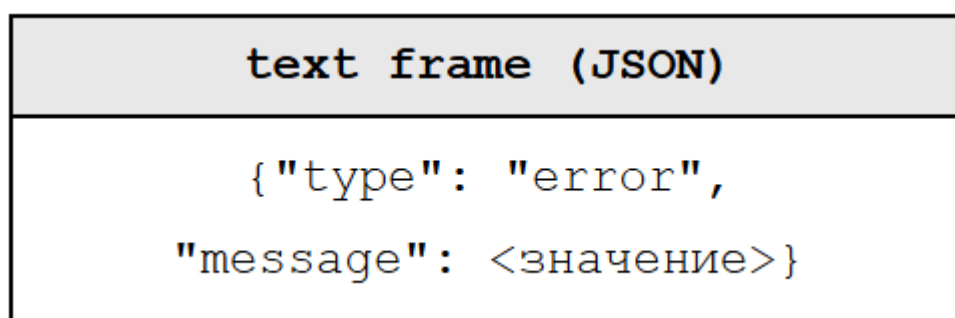


Рисунок 11 – Формат управляющего сообщения об ошибке

Сообщение передается в виде текстового фрагмента в формате JSON. Структура объекта приведена в таблице 12.

Таблица 12 — Формат сообщения `error`

Поле	Тип	Описание
<code>type</code>	строка UTF-8	Фиксированное значение <code>"error"</code> . Служит индикатором возникновения нештатной ситуации.
<code>message</code>	строка UTF-8	Текстовое описание ошибки, определяющее характер возникшей проблемы.

2 Разработка клиента и сервера

На данном этапе курсовой работы осуществляется реализация клиент-серверного взаимодействия, направленного на передачу файлов между программой-клиентом и программой-сервером. Основной целью данного этапа является реализация разработанного протокола прикладного уровня, обеспечивающего загрузку файлов на сервер, скачивание файлов с сервера, аутентификацию пользователей и протоколирование операций.

Для понимания реализации в первую очередь были определены необходимые функции программы-клиента и программы-сервера, перечень которых приведён в разделе 1.

Программа-клиент реализована в виде веб-приложения и выполняется в браузере пользователя. Программа-сервер запускается на и обеспечивает обработку запросов, хранение файлов и передачу клиентского приложения. Запуск осуществляется единой командой `npm run dev`, после чего программа-сервер становится доступна по адресу `http://localhost:3000`, а программа-клиент открывается в браузере по тому же адресу.

Программа-клиент состоит из следующих компонентов. Компонент `Login.tsx` реализует экран входа и регистрации: выполняет HTTP-аутентификацию посредством отправки запросов к REST API сервера, осуществляет периодическую проверку доступности сервера и сохраняет полученный токен в хранилище сессии браузера. Компонент `App.tsx` является корневым компонентом приложения и отвечает за маршрутизацию: в зависимости от роли аутентифицированного пользователя отображается либо панель обычного пользователя, либо панель администратора. Компонент `UserDashboard.tsx` является главным компонентом программы-клиента и содержит всю логику разработанного протокола: установку WebSocket-соединения, отправку запроса на авторизацию, загрузку и скачивание файлов, обработку ошибок сетевого взаимодействия и отображение прогресса передачи.

Компонент `AdminDashboard.tsx` реализует панель администратора: отображение статистики активных подключений и переданных данных, просмотр, редактирование и удаление записей журнала событий, просмотр списка передач с графиком скорости и перехватчиком пакетов.

`handleSubmit` (`Login.tsx`) Выполняет HTTP-аутентификацию пользователя. Отправляет POST-запрос на `/api/auth/login` с именем пользователя и паролем. При успешном ответе сохраняет полученный JWT-токен и данные пользователя в хранилище сессии браузера. При регистрации отправляет POST-запрос на `/api/auth/register` с дополнительным параметром роли пользователя. Реализована обработка ошибок сети и таймаута запроса.

`fetchFiles` (`UserDashboard.tsx`) Запрашивает у сервера актуальный список доступных для скачивания файлов посредством HTTP GET-запроса на `/api/files`. Вызывается при инициализации компонента, после успешной загрузки файла, а также при получении широковещательного сообщения `file_list_update`.

`UseEffect` – установка WebSocket-соединения (`UserDashboard.tsx`) Устанавливает WebSocket-соединение с сервером сразу после загрузки компонента. После установления соединения немедленно формирует и отправляет сообщение `AUTH` с JWT-токеном из хранилища сессии. Обрабатывает все входящие сообщения сервера: текстовые JSON-сообщения (`auth_success`, `file_success`, `file_list_update`, `error`) и бинарные сообщения протокола. При получении сообщений скачивания накапливает данные в буфере и отслеживает прогресс. При получении `DOWNLOAD_END` собирает накопленные фрагменты в файл и инициирует его сохранение на диск пользователя. Реализована обработка закрытия соединения с кодом 1008 (превышение лимита клиентов).

`uploadFile` (`UserDashboard.tsx`) Иницирует процесс загрузки

выбранного файла на сервер. Генерирует случайный идентификатор сессии передачи, формирует и отправляет сообщение `FILE_START` с идентификатором и метаданными файла в формате JSON, после чего передаёт управление функции `sendFileChunks`.

`sendFileChunks (UserDashboard.tsx)` Выполняет побайтовую передачу содержимого файла на сервер. Последовательно считывает файл фрагментами по 65 536 байт, формирует сообщение `FILE_CHUNK` с идентификатором сессии и данными фрагмента и отправляет его на сервер. Отслеживает прогресс передачи и реализует механизм управления потоком: при заполнении буфера отправки более чем на 5 МБ приостанавливает передачу на 50 миллисекунд. После отправки последнего фрагмента формирует и отправляет сообщение `FILE_END`.

`downloadFile (UserDashboard.tsx)` Отправляет на сервер запрос на скачивание файла. Формирует сообщение `DOWNLOAD_REQUEST` с четырёхбайтовым идентификатором запрошенного файла из базы данных сервера и передаёт его через установленное WebSocket-соединение.

Код программы-клиента представлен в приложении Б.

В программе-сервере реализованы следующие компоненты. Файл `server.ts` является главным файлом программы-сервера и содержит логику обработки всех входящих запросов: регистрацию и аутентификацию пользователей посредством REST API, приём WebSocket-подключений с ограничением в три одновременных соединения, обработку бинарных сообщений протокола, передачу файлов клиентам и широковебательную рассылку уведомлений. Файл `src/db.ts` реализует логику работы с базой данных SQLite: инициализацию схемы таблиц пользователей, журнала событий, передач и метрик, а также выполнение миграций при изменении структуры базы данных. Файл `src/worker.js` реализует фоновый процесс записи файлов на диск посредством Worker Thread, что исключает блокировку

основного потока сервера при операциях файлового ввода-вывода.

Обработчик `POST /api/auth/register` Принимает HTTP-запрос на регистрацию нового пользователя. Проверяет уникальность имени пользователя в базе данных, выполняет хэширование пароля с использованием алгоритма `bcrypt` и сохраняет нового пользователя в базу данных. При попытке зарегистрировать уже существующее имя пользователя возвращает сообщение об ошибке.

Обработчик `POST /api/auth/login` Принимает HTTP-запрос на аутентификацию пользователя. Выполняет поиск пользователя в базе данных по имени, проверяет соответствие переданного пароля сохранённому хэшу и при успешной проверке формирует и возвращает JWT-токен со сроком действия 24 часа. Записывает событие входа в журнал. При несоответствии учётных данных возвращает сообщение об ошибке.

Обработчик `GET /api/files` Принимает HTTP-запрос на получение списка файлов. Выполняет выборку из базы данных всех успешно завершённых передач, для которых файл присутствует на диске, и возвращает их клиенту в формате JSON.

Обработчик подключения WebSocket (`wss.on('connection')`) Принимает входящие WebSocket-соединения. Проверяет текущее число активных подключений: при превышении установленного ограничения в три одновременных соединения возвращает клиенту сообщение об ошибке и закрывает соединение с кодом 1008. При допустимом числе подключений регистрирует соединение, запускает механизм heartbeat-проверки и ожидает первого сообщения аутентификации.

Обработчик входящих сообщений WebSocket (`ws.on('message')`) Принимает и обрабатывает все входящие бинарные сообщения от программы-клиента. Определяет тип сообщения по первому байту и выполняет соответствующую операцию:

- при получении AUTH извлекает JWT-токен, выполняет его

верификацию и при успешной проверке устанавливает авторизованную сессию, отправляет клиенту подтверждение `auth_success` и записывает событие в журнал;

- при получении `FILE_START` извлекает идентификатор сессии и метаданные файла, создаёт запись о передаче в базе данных и направляет команду на открытие файлового потока фоновому процессу;

- при получении `FILE_CHUNK` извлекает идентификатор сессии и данные фрагмента, направляет их фоновому процессу для записи на диск и фиксирует метрики скорости передачи в базе данных;

- при получении `FILE_END` направляет фоновому процессу команду на закрытие файлового потока, обновляет статус передачи в базе данных, отправляет клиенту подтверждение `file_success` и рассылает широковещательное сообщение `file_list_update` всем подключённым клиентам;

- при получении `DOWNLOAD_REQUEST` выполняет поиск запрошенного файла в базе данных и на диске, при наличии файла отправляет клиенту сообщение `DOWNLOAD_START` с метаданными, затем последовательно считывает файл блоками по 65 536 байт и отправляет каждый блок как сообщение `DOWNLOAD_CHUNK`, после передачи последнего блока отправляет сообщение `DOWNLOAD_END` и записывает событие скачивания в журнал. При отсутствии файла возвращает клиенту сообщение об ошибке.

Механизм heartbeat Периодически с интервалом 10 секунд отправляет ping-сообщение каждому подключённому клиенту. Если клиент не ответил на предыдущую проверку, соединение принудительно закрывается посредством вызова `ws.terminate()`. Реализован для обнаружения и освобождения «мёртвых» соединений, не закрытых корректно.

Обработчик закрытия соединения (`ws.on('close')`) Вызывается при закрытии WebSocket-соединения. Удаляет соединение из множества активных

клиентов, записывает событие отключения в журнал и освобождает связанные ресурсы.

Обработчики REST API администратора Принимают HTTP-запросы от панели администратора и обеспечивают доступ к данным журнала событий, списку передач с детализацией метрик и пакетов, статистике активных подключений. Реализованы операции просмотра, редактирования и удаления записей журнала и передач, доступные только пользователям с ролью администратора.

Код программы-сервера представлен в приложении В.

3 Тестирование разработанных программ

Тестирование проводилось с использованием анализатора трафика Wireshark. Захват пакетов выполнялся на интерфейсе Loopback (localhost:3000). Приложение запускалось командой `npm run dev`. Для фильтрации трафика применялся фильтр `tcp.port == 3000`. Содержимое TCP-потоков получено через функцию «Анализ → Следовать → Поток TCP».

В ходе тестирования был зафиксирован процесс установления защищённого WebSocket-соединения между клиентом и сервером (Рисунок 12).

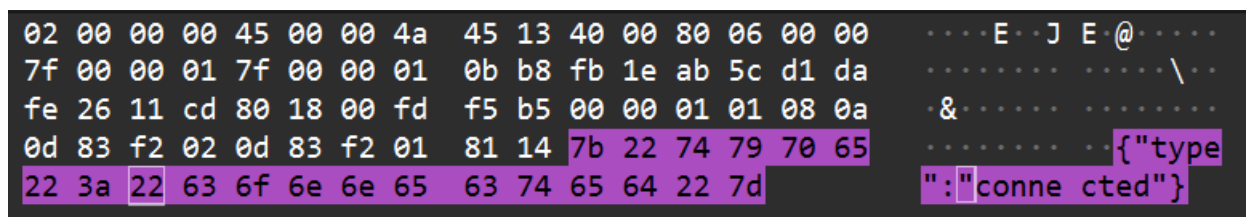


Рисунок 12 – Установление WebSocket-соединения

Клиент отправляет GET-запрос к серверу с параметром аутентификации `token` в URL и заголовками для перехода на WebSocket:

```
GET /?token=ggT6Yw1yHwNt HTTP/1.1
Host: localhost:3000
Upgrade: websocket
Origin: http://localhost:3000
Sec-WebSocket-Version: 13
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7
Sec-WebSocket-Key: buEZIG8pFc4gnUoiUm9oKQ==
Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits
Sec-WebSocket-Protocol: vite-hmr
```

Сервер подтверждает переход на WebSocket протокол кодом состояния 101:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: JXYET088KtiUtIb4ubYeK3Jv0G0=
Sec-WebSocket-Protocol: vite-hmr
```

После установления WebSocket-соединения сервер отправляет клиенту управляющее JSON-сообщение:

```
..{"type":"connected"}
```

Клиент выполняет HTTP POST-запрос к `/api/auth/login`, запрос

приведён на рисунке 14. При успехе JWT-токен сохраняется в `sessionStorage`, после чего устанавливается WebSocket-соединение и отправляется сообщение `AUTH` с токеном. Сервер возвращает `{"type": "auth_success"}`. Ответ сервера приведён на рисунке 15.

```
TCP      56 3000 → 64286 [ACK] Seq=186 Ack=593 Win=64768 Len=0 T
HTTP/J... 725 POST /api/auth/login HTTP/1.1 , JSON (application/js
TCP      56 3000 → 12148 [ACK] Seq=8441 Ack=3551 Win=61952 Len=0
```

Рисунок 13 – Выполнение POST-запроса

```
0000  02 00 00 00 45 00 00 4d 47 62 40 00 80 06 00 00  ....E..M Gb@....
0010  7f 00 00 01 7f 00 00 01 0b b8 2e 15 4e c9 8c 44  ....N..D
0020  84 f6 99 73 80 18 00 fd 98 ce 00 00 01 01 08 0a  ...s.....
0030  0d 84 b0 45 0d 84 b0 44 81 17 7b 22 74 79 70 65  ...E..D..{"type
0040  22 3a 22 61 75 74 68 5f 73 75 63 63 65 73 73 22  ": "auth_ success
0050  7d                                     }
```

Рисунок 14 – Успешная авторизация

Пользователь выбирает файл через интерфейс и нажимает «Start Upload». Клиент отправляет `FILE_START` с метаданными, затем `FILE_CHUNK` фрагментами по 64 КБ, по завершении – `FILE_END`. Сервер подтверждает приём сообщением `file_success` и рассылает `file_list_update` всем клиентам. Отправка сообщения приведена на рисунке 16.

```
..{"type": "file_success", "fileId": 691065}..{"type": "file_list_update"}..
```

```
0000  02 00 00 00 45 00 00 5d 46 c4 40 00 80 06 00 00  ....E..] F @....
0010  7f 00 00 01 7f 00 00 01 0b b8 6a 92 c1 85 99 c6  ....j
0020  ec d0 5f 9d 80 18 03 5c 60 2a 00 00 01 01 08 0a  ....\..*.....
0030  0d 84 59 7c 0d 84 59 48 81 27 7b 22 74 79 70 65  ..Y|..YH..{"type
0040  22 3a 22 66 69 6c 65 5f 73 75 63 63 65 73 73 22  ": "file_ success
0050  2c 22 66 69 6c 65 49 64 22 3a 36 39 31 30 36 35  , "fileId ": 691065
0060  7d                                     }
```

Рисунок 15 – Обновление списка файлов

```
0000  02 00 00 00 45 00 00 51 46 c6 40 00 80 06 00 00  ....E..Q F @....
0010  7f 00 00 01 7f 00 00 01 0b b8 6a 92 c1 85 99 ef  ....j
0020  ec d0 5f 9d 80 18 03 5c a1 33 00 00 01 01 08 0a  ....\..3.....
0030  0d 84 59 7c 0d 84 59 7c 81 1b 7b 22 74 79 70 65  ..Y|..Y|..{"type
0040  22 3a 22 66 69 6c 65 5f 6c 69 73 74 5f 75 70 64  ": "file_ list_upd
0050  61 74 65 22 7d                         ate"}
```

Рисунок 16 – Успешная отправка файла

Обработчик `POST /api/auth/register` проверяет уникальность имени,

хэширует пароль через bcrypt и сохраняет пользователя в БД. Обработчик POST /api/auth/login проверяет учётные данные и возвращает JWT-токен сроком 24 часа. Все события фиксируются в журнале, рисунок 17. Фрагмент с отправляемыми данными пользователя приведён на рисунке 18.

```
TCP      56 12148 → 3000 [ACK] Seq=12039 Ack=11123 Win=54272 Len=0
HTTP/J... 729 POST /api/auth/login HTTP/1.1 , JSON (application/json)
TCP      56 3000 → 12148 [ACK] Seq=11123 Ack=12712 Win=52736 Len=0
```

Рисунок 17 – Запрос авторизации

```
0280  2d 4c 61 6e 67 75 61 67 65 3a 20 72 75 2d 52 55 -Languag e: ru-RU
0290  2c 72 75 3b 71 3d 30 2e 39 2c 65 6e 2d 55 53 3b ,ru;q=0.9,en-US;
02a0  71 3d 30 2e 38 2c 65 6e 3b 71 3d 30 2e 37 0d 0a q=0.8,en;q=0.7
02b0  0d 0a 7b 22 75 73 65 72 6e 61 6d 65 22 3a 22 75 {"user name":"u
02c0  73 65 72 31 22 2c 22 70 61 73 73 77 6f 72 64 22 ser1","p assword"
02d0  3a 22 75 73 65 72 31 22 7d : "user1" }
```

Рисунок 18 – Данные для авторизации

При попытке установить четвёртое соединение сервер немедленно возвращает сообщение об ошибке и закрывает соединение с кодом 1008. Результат приведён на рисунке 19.

```
0000  02 00 00 00 45 00 00 53 48 87 40 00 80 06 00 00 ... E S H @ ...
0010  7f 00 00 01 7f 00 00 01 0b b8 1b 01 dc dd 40 3c ... @ <
0020  91 20 49 20 80 18 00 fd bc 44 00 00 01 01 08 0a ... I ... D ...
0030  0d 85 47 8f 0d 85 47 8f 88 1d 03 f0 53 65 72 76 ... G ... G ... Serv
0040  65 72 20 66 75 6c 6c 20 28 6d 61 78 20 33 20 63 er full (max 3 c
0050  6c 69 65 6e 74 73 29 ... lients)
```

Рисунок 19 – Максимальное количество клиентов

Каждые 10 секунд сервер отправляет WebSocket Ping-фрейм каждому клиенту. При отсутствии ответа соединение принудительно закрывается через ws.terminate(). Ping-фреймы наблюдались в захваченном трафике в виде однобайтовых управляющих фреймов между полезными данными (Рисунок 20).

```
WebSoc... 81 WebSocket Text [FIN]
WebSoc... 58 WebSocket Ping [FIN]
WebSoc... 62 WebSocket Pong [FIN] [MASKED]
```

Рисунок 20 – Мониторинг состояния сервера

4 Разработка руководства пользователя

Было разработано руководство предназначено для пользователей системы передачи файлов, разработанной в рамках курсовой работы.

Программа клиент организует ролевую модель доступа для двух типов пользователей:

- пользователь: может скачивать файлы с сервера;
- администратор: обладает правами просмотра, редактирования и удаления журналов событий, историй передач файлов и статистикой передачи.

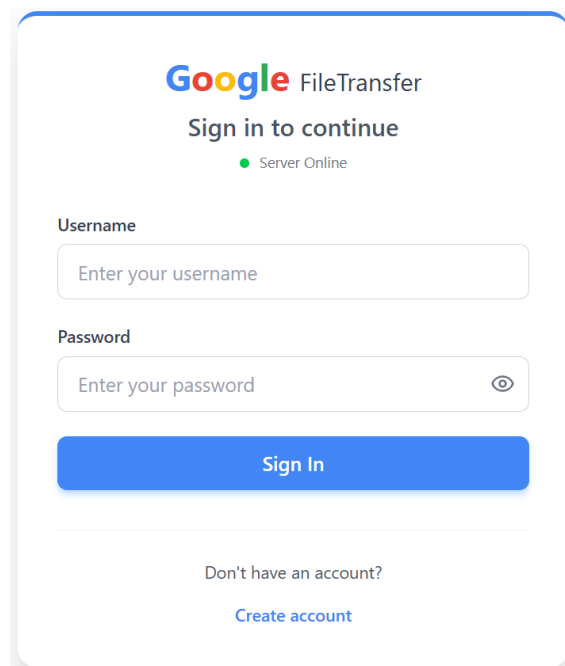
Программа клиент реализует следующие функции:

- установление соединения с программой сервером;
- проведение авторизации пользователя;
- отправка пользователю выбранного файла;
- предоставление, редактирование и удаление журналов событий;
- закрытие соединения.

Запуск программы-клиента осуществляется переходом пользователя по адресу, на котором сервер открыл соединение `http://localhost:3000/`. Это происходит автоматически при запуске программы сервера. После перехода по выданному адресу перед пользователем откроется окно авторизации, в которое необходимо ввести логин и пароль пользователя. Форма авторизации приведена на рисунке 21.

После успешной авторизации перед обычным пользователем открывается окно выбора файла, в котором пользователь с помощью программы проводника выбирает необходимый ему файл. Окно выбора файла приведён на рисунке 22.

После авторизации за администратора перед пользователем открывается панель администратора, на которой приведено количество активных подключений, количество отправленных файлов, журнал аудита, журнала длительности передачи и списка отправки файлов. Панель администратора приведена на рисунке 23.



Google FileTransfer

Sign in to continue

● Server Online

Username

Enter your username

Password

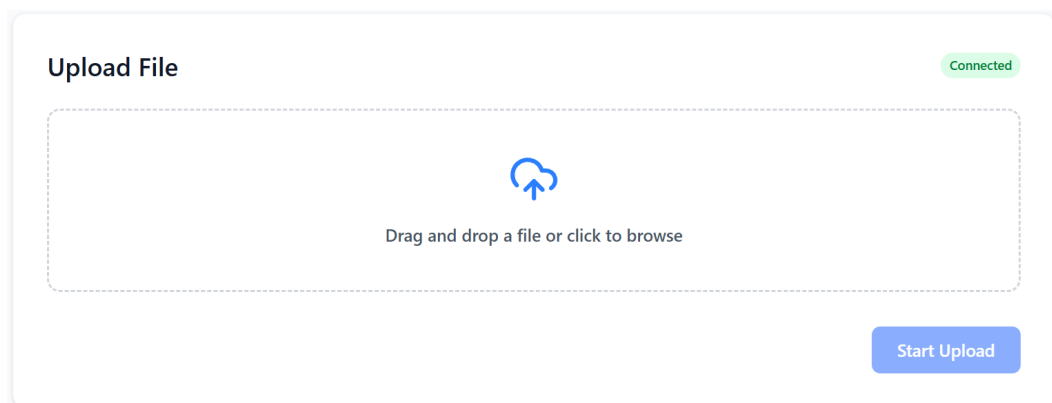
Enter your password

Sign In

Don't have an account?

[Create account](#)

Рисунок 21 – Форма авторизации пользователя



Upload File

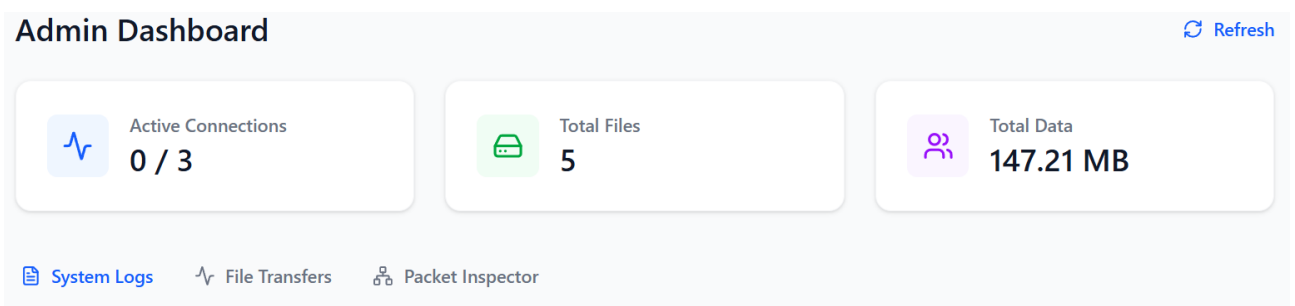
Connected

Drag and drop a file or click to browse

Start Upload

Рисунок 22 – Окно выбора файла

Программа сервер запускается командой `npm run dev` и работает в автономном режиме, обрабатывая действия пользователей. Результат запуска программы клиента приведён на рисунке .



Admin Dashboard

Refresh

Active Connections 0 / 3	Total Files 5	Total Data 147.21 MB
-----------------------------	------------------	-------------------------

System Logs File Transfers Packet Inspector

Рисунок 23 – Панель администратора

Заключение

В курсовой работе был подробно рассмотрен процесс проектирования, реализации тестирования приложения с сетевым взаимодействием.

Был разработан протокол прикладного уровня, работающий на основе протокола транспортного уровня TCP.

Была разработана программа-сервер, осуществляющая многопоточную работу нескольких одновременно подключённых программ клиентов. Были реализованы функции создания сетевых сокетов, обмен сообщениями между сетевыми узлами, отправка выбранного файла с помощью сегментирования его на большое количество равных частей, обработкой возникающих в ходе работы ошибок.

Была реализована программа-клиент, обладающая ролевой моделью разграничения доступа, выполняющая авторизацию пользователя, предоставление возможности выбора отправляемого файла, мониторинга количества подключений, просмотра журналов аудита, длительности отправки файлов, подробные логи и процессе сегментирования и отправки файла, количестве отправленных файлов.

Была проверена корректности разработанного приложения при помощи программы анализатора трафика компьютерных сетей Wireshark.

Было разработано руководство пользователя, в котором объясняется как пользователю взаимодействовать с обеими сторонами приложения для отправки данных.

Таким образом, курсовая работа удовлетворяет всем требованиям технического задания и была выполнена в полном объёме.

Список используемых источников

- 1 ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – 27 с.
- 2 ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : Стандартинформ, 2018. – 124 с.
- 3 Таненбаум, Э. Компьютерные сети / Э. Таненбаум, Д. Уэзеролл. – 6-е изд. – Санкт-Петербург : Питер, 2023. – 992 с.
- 4 Олифер, В. Г. Компьютерные сети. Принципы, технологии, протоколы : учебник для вузов / В. Г. Олифер, Н. А. Олифер. – 6-е изд. – Санкт-Петербург : Питер, 2020. – 1008 с.
- 5 Куроуз, Дж. Компьютерные сети. Настольная книга системного администратора / Дж. Куроуз, К. Росс. – 8-е изд. – Москва : Эксмо, 2022. – 960 с.
- 6 Каскиаро, М. Шаблоны проектирования Node.js / М. Каскиаро, Л. Маммино. – Санкт-Петербург : Питер, 2021. – 672 с.
- 7 Шмитт, К. Построение приложений на React / К. Шмитт. – Санкт-Петербург : Питер, 2023. – 320 с.

Приложение А
(обязательное)

Диаграммы состояний и последовательности

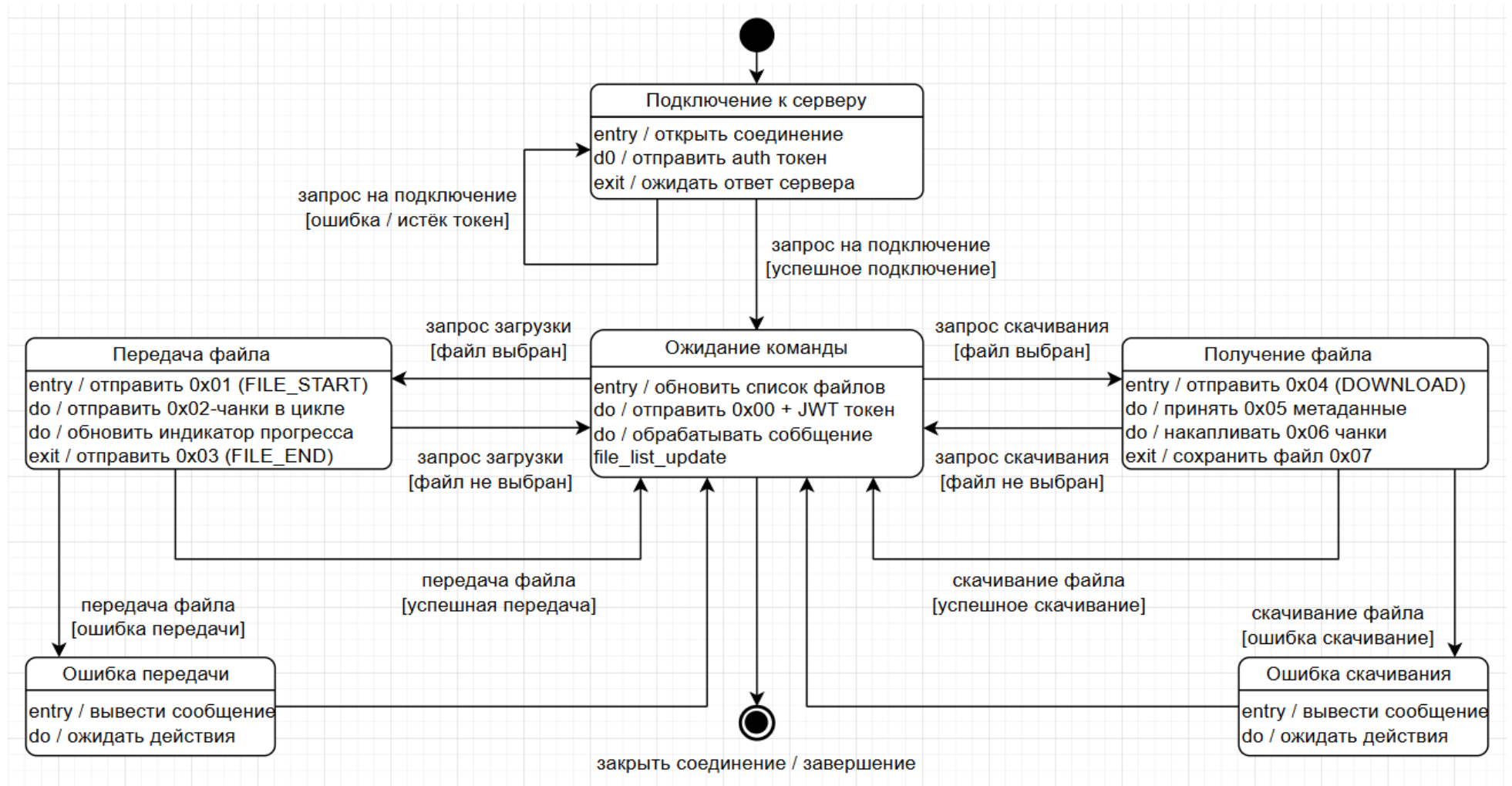


Рисунок А.1 – Диаграмма состояний программы-клиента

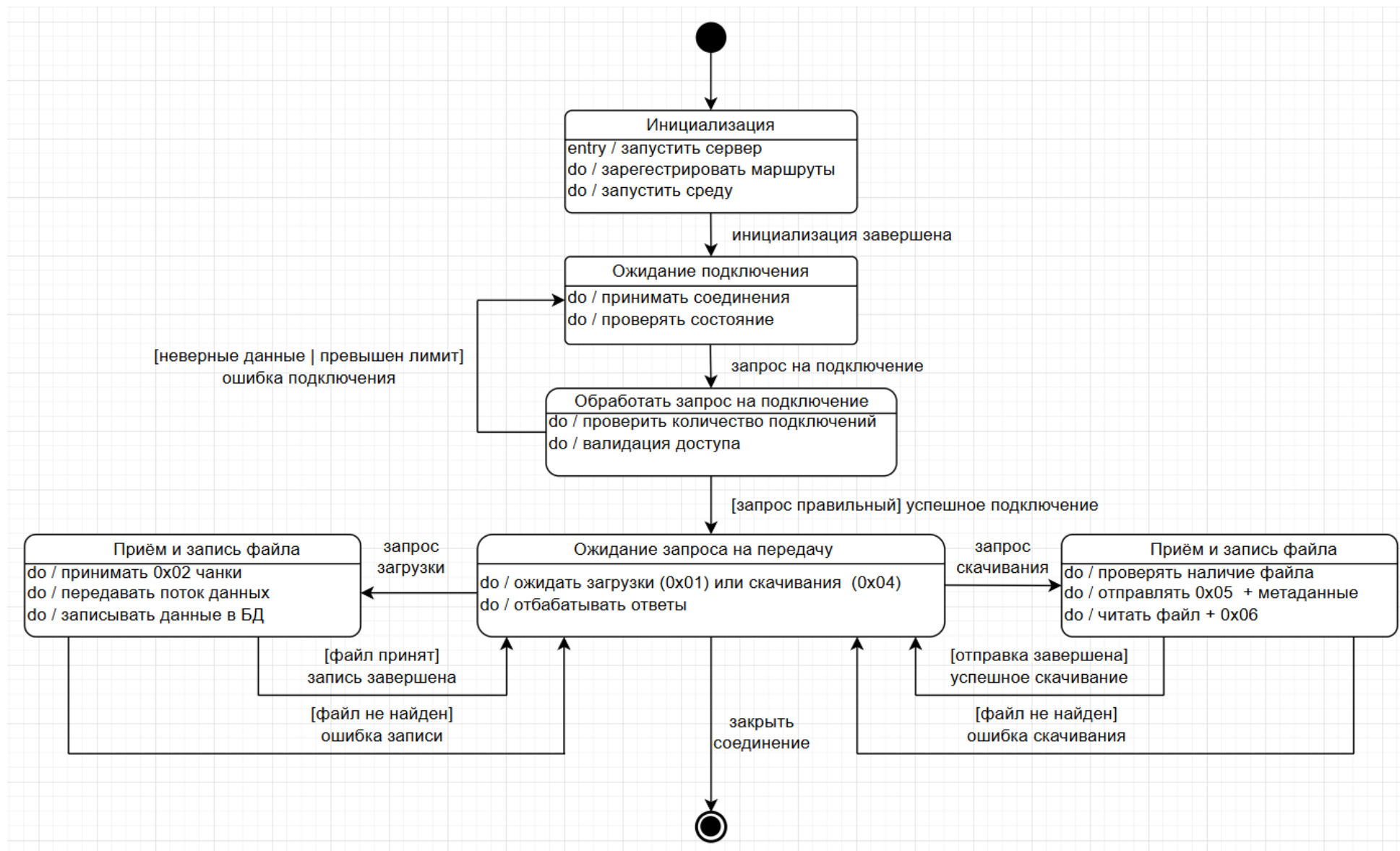


Рисунок А.2 – Диаграмма состояний программы-сервера

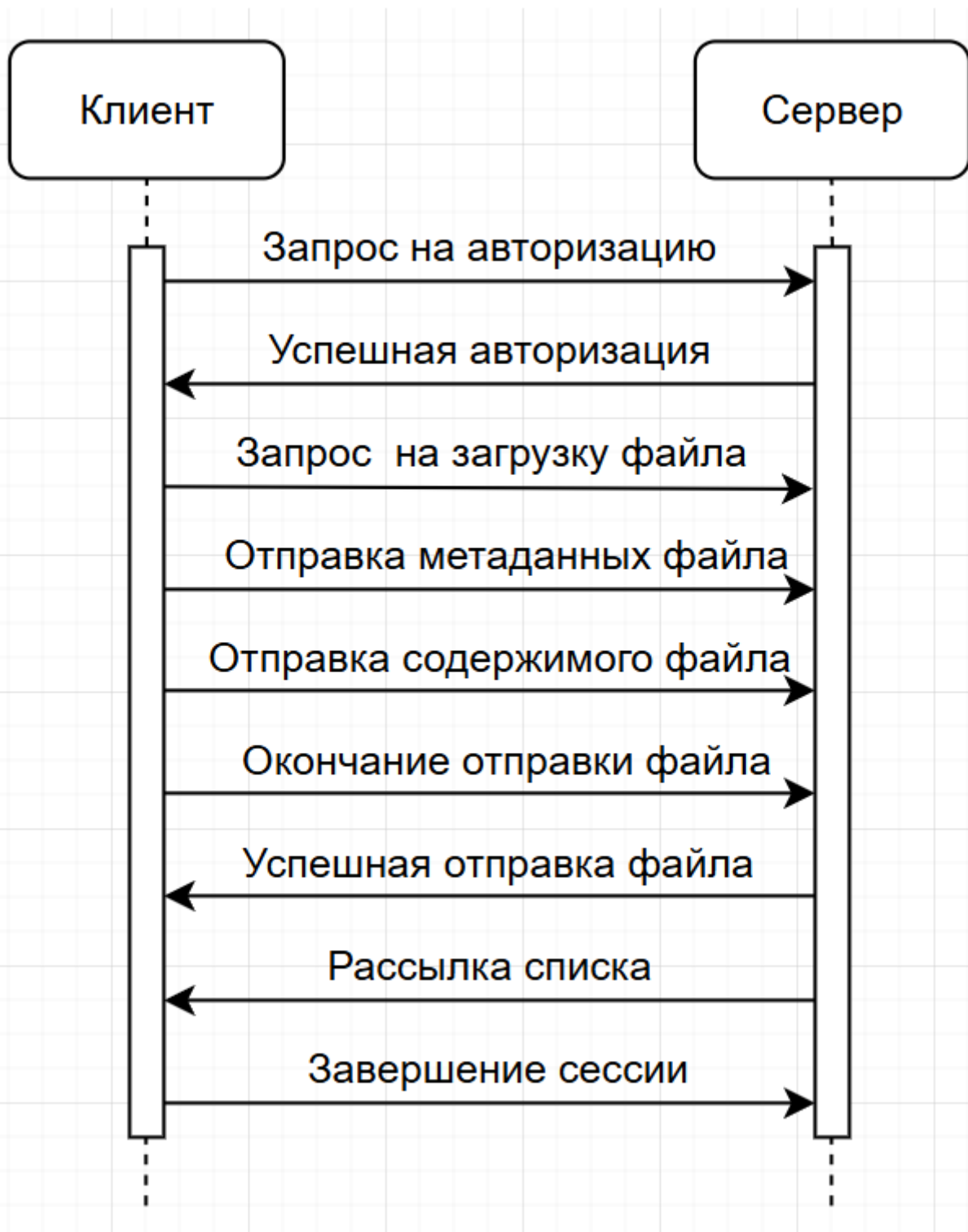


Рисунок А.3 – Диаграмма последовательности: успешная загрузка файла

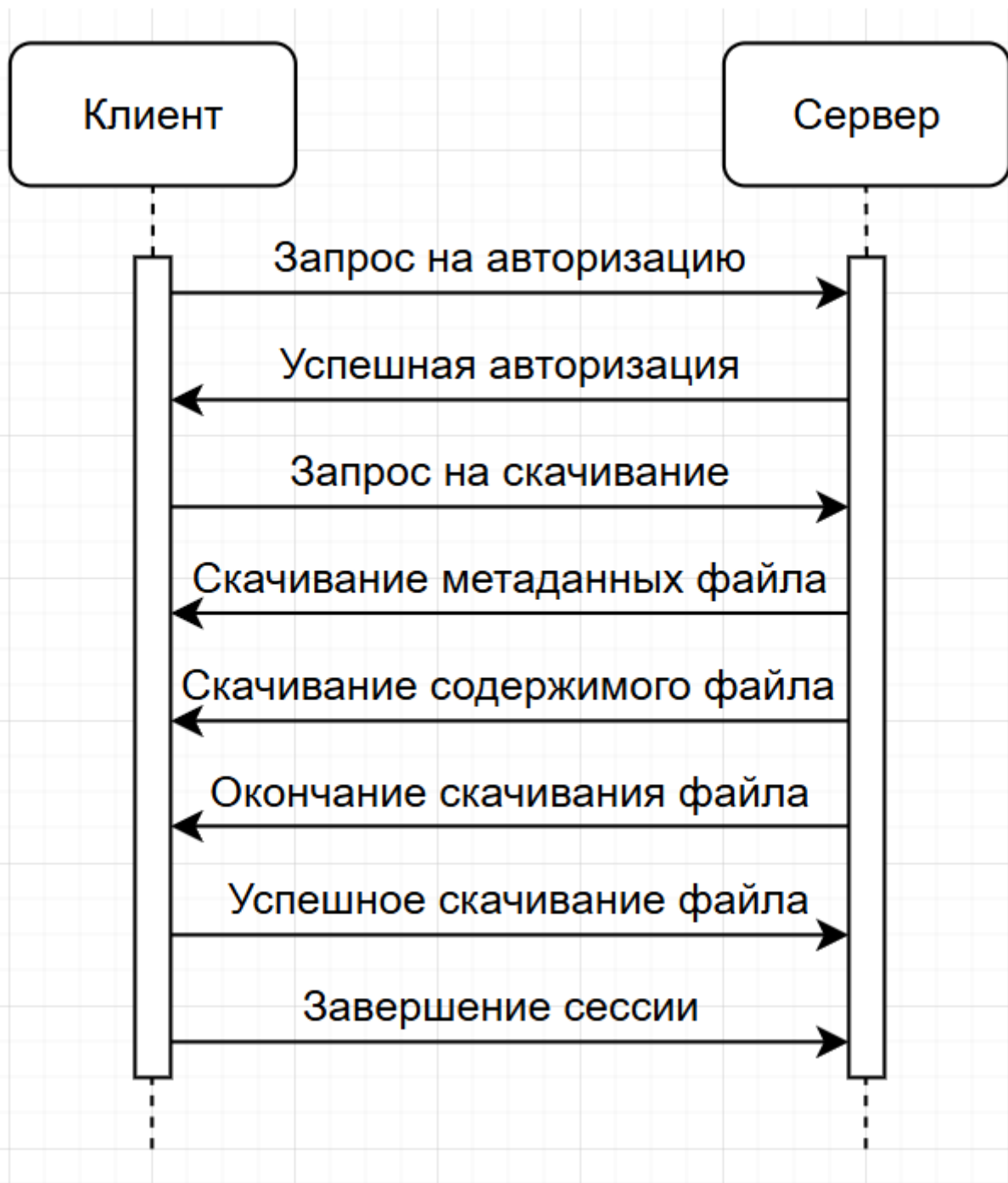


Рисунок А.4 – Диаграмма последовательности: успешное скачивание файла

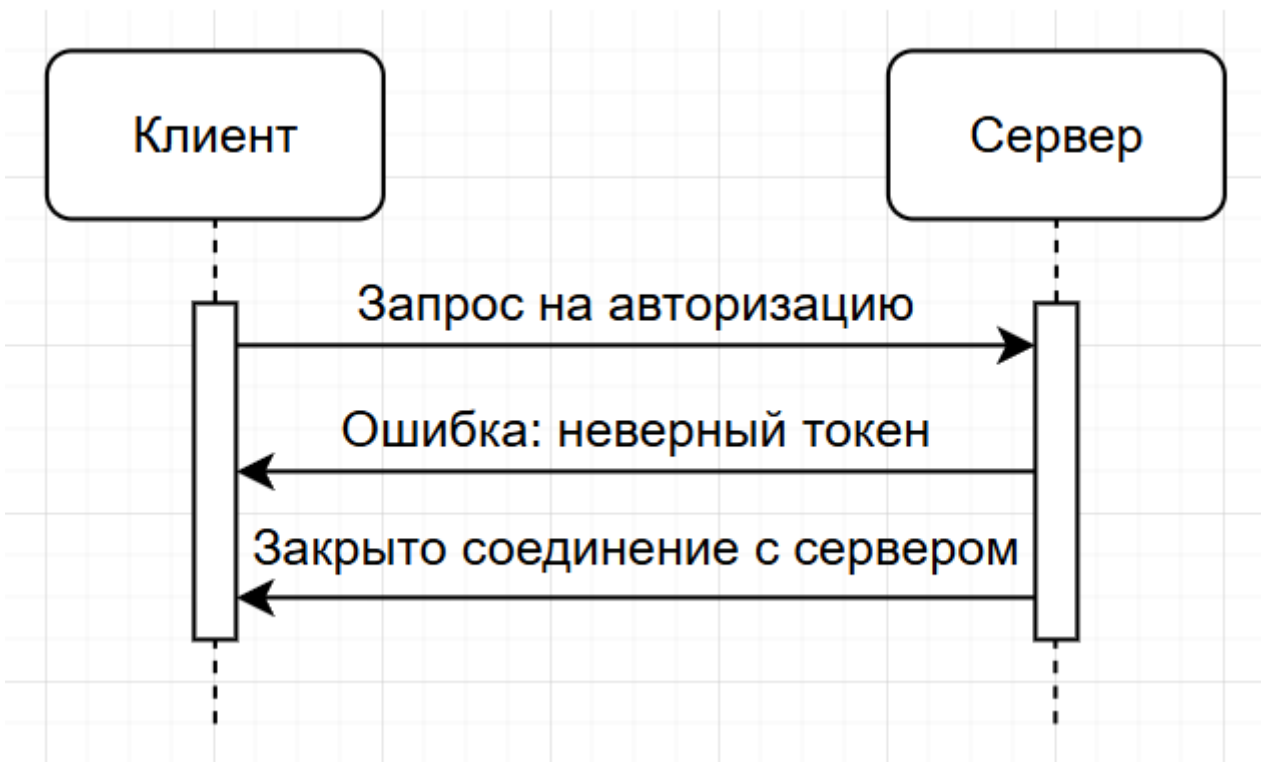


Рисунок А.5 – Диаграмма последовательности: ошибка аутентификации

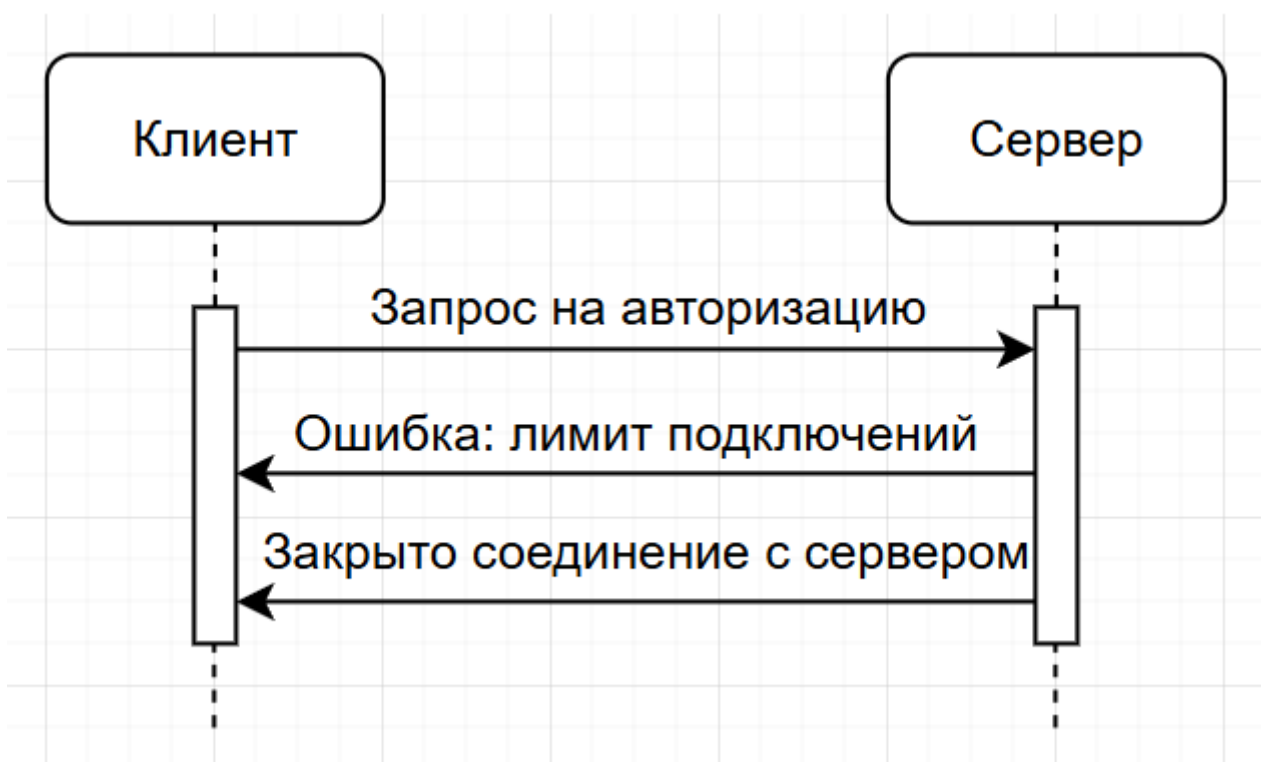


Рисунок А.6 – Диаграмма последовательности: превышение лимита
подключений

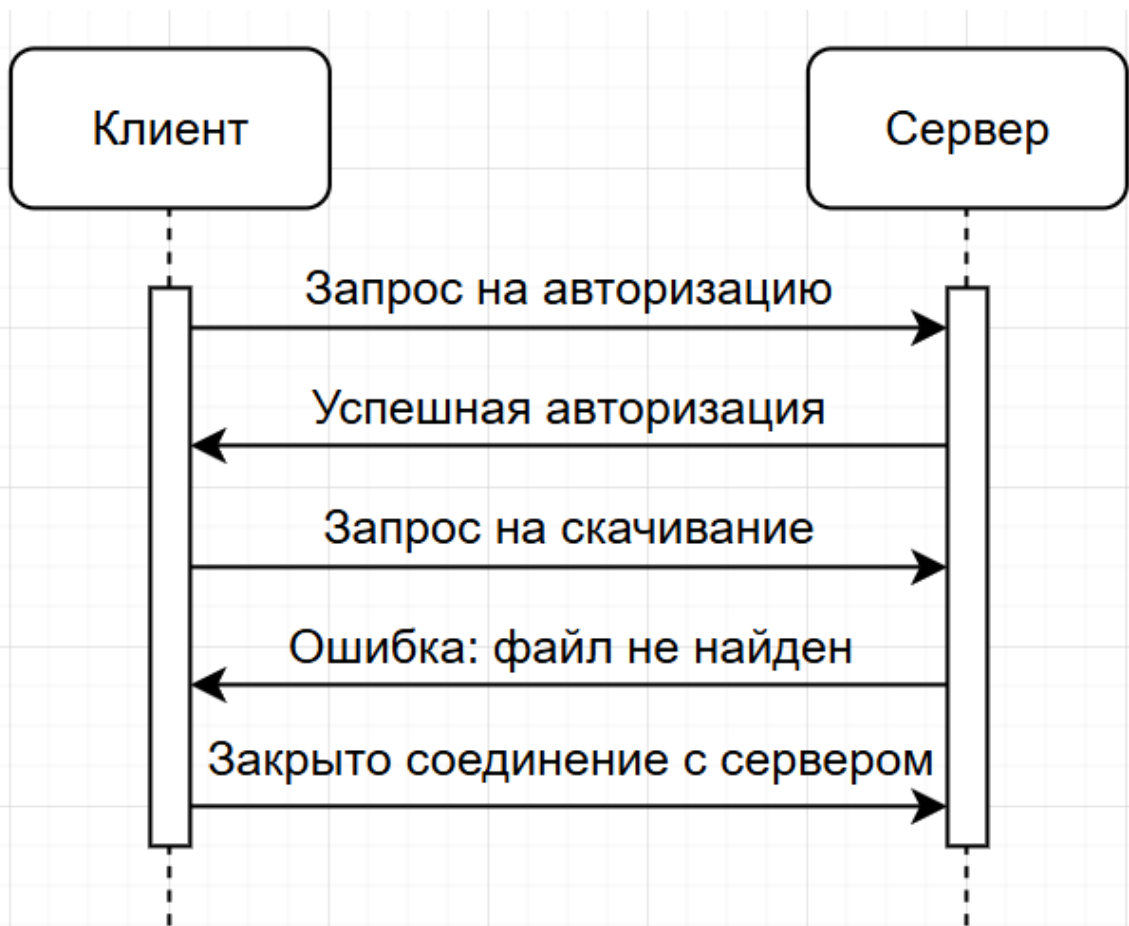


Рисунок А.7 – Диаграмма последовательности: запрос несуществующего файла

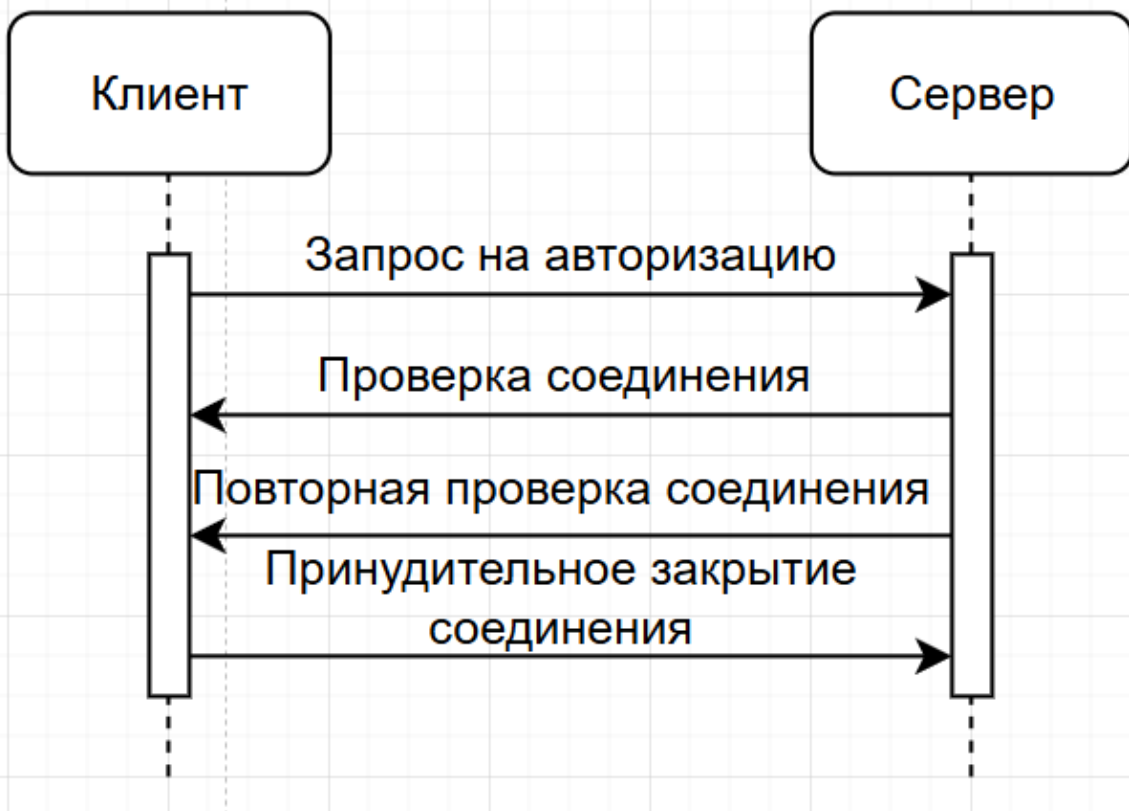


Рисунок А.8 – Диаграмма последовательности: принудительное закрытие

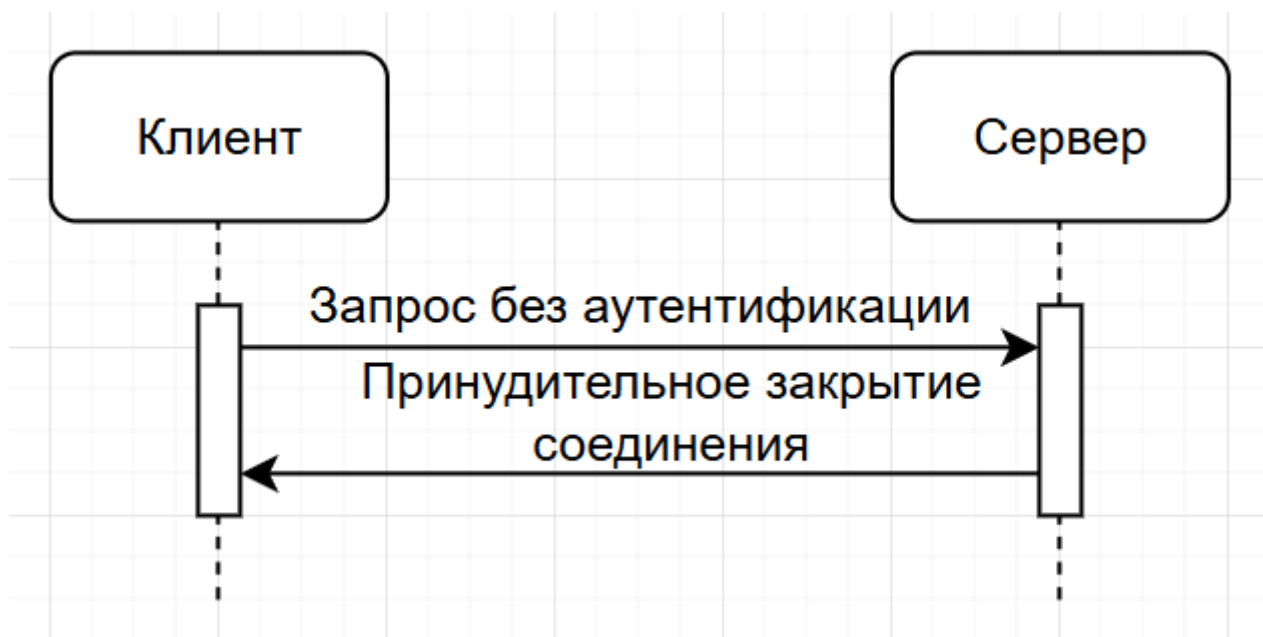


Рисунок А.9 – Диаграмма последовательности: несанкционированная команда без аутентификации

Приложение Б

(обязательное)

Код программы-клиента

```
// =====  
  
// UserDashboard.tsx – Главный компонент программы-клиента.  
// Реализует всю логику протокола прикладного уровня поверх  
// WebSocket: аутентификацию, загрузку и скачивание файлов,  
// а также обработку ошибок сетевого взаимодействия.  
// =====  
  
import React, { useState, useRef, useEffect } from 'react';  
import { UploadCloud, CheckCircle, AlertCircle, Download, File, RefreshCw } from  
'lucide-react';  
  
// Компонент принимает объект текущего пользователя в качестве параметра  
export default function UserDashboard({ user }: { user: any }) {  
  
  // --- Состояние загрузки файла ---  
  const [file, setFile] = useState<File | null>(null);           // Выбранный  
для загрузки файл  
  const [progress, setProgress] = useState(0);                 // Прогресс  
загрузки в процентах (0–100)  
  const [status, setStatus] = useState<                               // Текущее  
состояние передачи  
    'idle' | 'connecting' | 'uploading' | 'success' | 'error'  
  >('idle');  
  const [errorMsg, setErrorMsg] = useState('');                // Текст  
сообщения об ошибке  
  const [isConnected, setIsConnected] = useState(false);       // Флаг  
установленного соединения с сервером  
  const [availableFiles, setAvailableFiles] = useState<any[]>([]); // Список  
файлов, доступных для скачивания  
  const [isRefreshing, setIsRefreshing] = useState(false);     // Флаг  
обновления списка файлов  
  
  // Ссылки на WebSocket и идентификатор текущей сессии передачи.  
  // Используются ref, а не state, чтобы избежать лишних перерисовок  
  // и гарантировать актуальность значения внутри колбэков WebSocket.  
  const wsRef = useRef<WebSocket | null>(null);  
  const fileIdRef = useRef<number>(0);  
  
  // --- Состояние скачивания файла ---  
  const [downloadProgress, setDownloadProgress] = useState(0);   //  
Прогресс скачивания в процентах  
  const [downloadingFile, setDownloadingFile] = useState<string | null>(null);  
// Имя скачиваемого файла  
  const downloadBufferRef = useRef<Uint8Array[]>([]);           //  
Буфер накопленных чанков файла
```

```

const downloadMetaRef = useRef<{                                     //
  Метаданные скачиваемого файла
  filename: string;
  size: number;
} | null>(null);

// =====
// useEffect – инициализация WebSocket-соединения при монтировании
// компонента. Выполняется один раз после первой отрисовки.
// =====
useEffect(() => {
  // Запрашиваем список файлов сразу при открытии страницы
  fetchFiles();

  // Получаем JWT-токен из хранилища сессии браузера.
  // Токен был сохранён при успешном входе в компоненте Login.
  const token = sessionStorage.getItem('token');

  // Определяем протокол WebSocket в зависимости от протокола страницы:
  // ws:// для HTTP, wss:// для HTTPS
  const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
  const wsUrl = `${protocol}://${window.location.host}/api/ws`;

  // Задержка 300 мс необходима для корректной работы в режиме React Strict
Mode,
  // при котором эффекты могут вызываться дважды подряд. Задержка позволяет
// предыдущему соединению успеть закрыться перед открытием нового.
  const connectTimeout = setTimeout(() => {
    const ws = new WebSocket(wsUrl);
    wsRef.current = ws;

    // Указываем браузеру принимать данные как ArrayBuffer,
    // что необходимо для корректной обработки бинарных сообщений протокола
    ws.binaryType = 'arraybuffer';

    // -----
    // Обработчик события открытия соединения.
    // Сразу после установления соединения отправляем AUTH (0x00).
    // -----
    ws.onopen = () => {
      if (!token) return; // Если токен отсутствует, не отправляем запрос

      // Кодировем JWT-токен в массив байт UTF-8
      const tokenBytes = new TextEncoder().encode(token);

      // Формируем бинарное сообщение AUTH:
      // [0x00] [токен в байтах]
      // 1 байт N байт
      const authMsg = new Uint8Array(1 + tokenBytes.length);
      authMsg[0] = 0x00; // Опкод AUTH
      authMsg.set(tokenBytes, 1); // Токен начиная с байта 1
    }
  }, 300);
}, [token]);

```

```

ws.send(authMsg);
};

// -----
// Обработчик входящих сообщений от сервера.
// Сервер отправляет два типа сообщений:
//   - текстовые JSON (управляющие ответы)
//   - бинарные ArrayBuffer (данные файла при скачивании)
// -----
ws.onmessage = async (event) => {

  if (typeof event.data === 'string') {
    // --- Обработка текстовых JSON-сообщений от сервера ---
    const data = JSON.parse(event.data);

    if (data.type === 'auth_success') {
      // Аутентификация прошла успешно – соединение авторизовано
      setIsConnected(true);
    } else if (data.type === 'error') {
      // Сервер вернул ошибку
      setStatus('error');
      setErrorMsg(data.message);
      setDownloadingFile(null);

      // Если ошибка связана с переполнением сервера –
      // закрываем соединение, дальнейшая работа невозможна
      if (data.message.includes('full')) {
        ws.close();
      }
    } else if (data.type === 'file_success') {
      // Сервер подтвердил успешный приём и сохранение файла
      setStatus('success');
      fetchFiles(); // Обновляем список файлов
    } else if (data.type === 'file_list_update') {
      // Другой клиент загрузил новый файл – обновляем список
      fetchFiles();
    }
  } else if (event.data instanceof ArrayBuffer) {
    // --- Обработка бинарных сообщений протокола ---
    // Читаем первый байт – опкод типа сообщения
    const view = new DataView(event.data);
    const type = view.getUint8(0);

    if (type === 0x05) {
      // DOWNLOAD_START: сервер передаёт метаданные файла
      // Байты начиная с 1-го – JSON с полями filename и size
      const metaBytes = new Uint8Array(event.data.slice(1));
    }
  }
};

```

```

const metaStr = new TextDecoder().decode(metaBytes);
downloadMetaRef.current = JSON.parse(metaStr);

// Очищаем буфер перед началом приёма новых данных
downloadBufferRef.current = [];
setDownloadProgress(0);

// Отображаем имя скачиваемого файла в интерфейсе
if (downloadMetaRef.current) {
    setDownloadingFile(downloadMetaRef.current.filename);
}

} else if (type === 0x06) {
    // DOWNLOAD_CHUNK: очередной фрагмент данных файла
    // Байты начиная с 1-го – бинарные данные фрагмента
    const chunk = new Uint8Array(event.data.slice(1));
    downloadBufferRef.current.push(chunk); // Накапливаем в буфере

    // Вычисляем и обновляем прогресс скачивания
    if (downloadMetaRef.current) {
        const currentSize = downloadBufferRef.current.reduce(
            (acc, val) => acc + val.length, 0
        );
        setDownloadProgress(
            Math.round((currentSize / downloadMetaRef.current.size) * 100)
        );
    }

} else if (type === 0x07) {
    // DOWNLOAD_END: передача завершена, собираем файл из буфера
    // Объединяем все накопленные чанки в единый Blob
    const blob = new Blob(downloadBufferRef.current);

    // Создаём временную ссылку и программно инициируем скачивание
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = downloadMetaRef.current?.filename || 'downloaded_file';
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);

    // Освобождаем временный URL из памяти браузера
    URL.revokeObjectURL(url);

    // Сбрасываем состояние скачивания
    setDownloadingFile(null);
    setDownloadProgress(0);
    downloadBufferRef.current = [];
}
}

```

```

};

// -----
// Обработчик ошибки соединения.
// Вызывается при невозможности установить или поддерживать соединение.
// -----
ws.onerror = () => {
  if (!isConnected) {
    setStatus('error');
    setErrorMsg('Connection failed. Server might be full (max 3
clients).');
  }
};

// -----
// Обработчик закрытия соединения.
// Код 1008 означает, что сервер отклонил подключение
// из-за превышения лимита одновременных клиентов (max 3).
// -----
ws.onclose = (e) => {
  setIsConnected(false);
  if (e.code === 1008) {
    setStatus('error');
    setErrorMsg('Server full (max 3 clients). Please try again later.');
```

```

    } catch (e) {
        console.error('Failed to fetch files', e);
    } finally {
        setIsRefreshing(false);
    }
};

// =====
// downloadFile – отправляет запрос на скачивание файла (0x04).
// Принимает идентификатор записи файла в базе данных сервера.
// =====
const downloadFile = (id: number) => {
    // Проверяем наличие активного авторизованного соединения
    if (!wsRef.current || !isConnected) {
        alert('Not connected to server');
        return;
    }

    // Формируем бинарное сообщение DOWNLOAD_REQUEST:
    // [0x04] [fileId: 4 байта uint32 LE]
    const reqMsg = new Uint8Array(5);
    reqMsg[0] = 0x04; // Опкод DOWNLOAD_REQUEST
    new DataView(reqMsg.buffer).setUint32(1, id, true); // fileId в Little
    wsRef.current.send(reqMsg);
};

// =====
// handleFileChange – обработчик выбора файла пользователем.
// Сохраняет файл в состояние и сбрасывает предыдущий прогресс.
// =====
const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    if (e.target.files && e.target.files[0]) {
        setFile(e.target.files[0]);
        setStatus('idle');
        setProgress(0);
    }
};

// =====
// uploadFile – иницирует процесс загрузки файла на сервер.
// Генерирует идентификатор сессии, формирует и отправляет
// сообщение FILE_START (0x01), затем запускает sendFileChunks.
// =====
const uploadFile = () => {
    // Проверяем что файл выбран и соединение установлено
    if (!file || !wsRef.current || !isConnected) {
        if (!isConnected) setErrorMsg('Not connected to server. Refresh page.');
```

```

setStatus('uploading');

// Генерируем случайный идентификатор сессии передачи (1-999999).
// Используется для сопоставления чанков с конкретной передачей.
fileIdRef.current = Math.floor(Math.random() * 1000000);

// Формируем JSON-метаданные файла
const metadata = JSON.stringify({ filename: file.name, size: file.size });
const metaBytes = new TextEncoder().encode(metadata);

// Формируем бинарное сообщение FILE_START:
// [0x01] [fileId: 4 байта uint32 LE] [metadata: JSON в байтах]
const startMsg = new Uint8Array(5 + metaBytes.length);
startMsg[0] = 0x01; // Опкод FILE_START
new DataView(startMsg.buffer).setUint32(1, fileIdRef.current, true); //
fileId
startMsg.set(metaBytes, 5); // Метаданные начиная с байта 5
wsRef.current.send(startMsg);

// Запускаем потоковую передачу данных файла
sendFileChunks(wsRef.current, file, fileIdRef.current);
};

// =====
// sendFileChunks – выполняет побайтовую передачу файла на сервер.
// Читает файл фрагментами по 64 КБ, формирует FILE_CHUNK (0x02)
// для каждого фрагмента, по завершении отправляет FILE_END (0x03).
// =====
const sendFileChunks = async (ws: WebSocket, file: File, fileId: number) => {
  const CHUNK_SIZE = 1024 * 64; // Размер одного фрагмента: 64 КБ
  let offset = 0; // Текущая позиция чтения в файле

  // Вспомогательная функция асинхронного чтения фрагмента файла.
  // Использует FileReader API браузера для чтения blob в ArrayBuffer.
  const readChunk = (offset: number, size: number): Promise<ArrayBuffer> => {
    return new Promise((resolve, reject) => {
      const reader = new FileReader();
      reader.onload = (e) => resolve(e.target?.result as ArrayBuffer);
      reader.onerror = reject;
      reader.readAsArrayBuffer(file.slice(offset, offset + size));
    });
  };

  // Цикл передачи: продолжаем пока не передан весь файл
  while (offset < file.size) {
    // Прерываем передачу если соединение было закрыто
    if (ws.readyState !== WebSocket.OPEN) break;

    // Считываем очередной фрагмент файла
    const chunk = await readChunk(offset, CHUNK_SIZE);
  }
}

```

```

    // Формируем бинарное сообщение FILE_CHUNK:
    // [0x02] [fileId: 4 байта uint32 LE] [данные фрагмента]
    const chunkMsg = new Uint8Array(5 + chunk.byteLength);
    chunkMsg[0] = 0x02; // Опкод FILE_CHUNK
    new DataView(chunkMsg.buffer).setUint32(1, fileId, true); // fileId
    chunkMsg.set(new Uint8Array(chunk), 5); // Данные начиная с байта 5
    ws.send(chunkMsg);

    // Обновляем позицию чтения и прогресс в интерфейсе
    offset += chunk.byteLength;
    setProgress(Math.round((offset / file.size) * 100));

    // Механизм управления потоком: если буфер отправки WebSocket
    // превысил 5 МБ, делаем паузу 50 мс чтобы дать браузеру
    // возможность передать накопленные данные и освободить буфер
    if (ws.bufferedAmount > 1024 * 1024 * 5) {
        await new Promise(resolve => setTimeout(resolve, 50));
    }
}

// После отправки всех фрагментов отправляем FILE_END (0x03).
// Сигнализирует серверу о завершении передачи данного файла.
if (ws.readyState === WebSocket.OPEN) {
    const endMsg = new Uint8Array(5);
    endMsg[0] = 0x03; // Опкод FILE_END
    new DataView(endMsg.buffer).setUint32(1, fileId, true); // fileId
    ws.send(endMsg);
}
};

// =====
// Возвращаемая разметка компонента
// =====
return (
    <div className="space-y-6">
        {/* Блок загрузки файла */}
        <div className="bg-white p-8 rounded-xl shadow-sm border border-gray-100">
            <h2 className="text-2xl font-medium text-gray-900 mb-6 flex justify-
between items-center">
                Upload File
                {/* Индикатор состояния соединения с сервером */}
                <span className={`text-xs px-2 py-1 rounded-full ${
                    isConnected ? 'bg-green-100 text-green-700' : 'bg-red-100 text-red-
700'
                }`}>
                    {isConnected ? 'Connected' : 'Disconnected'}
                </span>
            </h2>

            {/* Зона drag-and-drop для выбора файла */}
            <div className="border-2 border-dashed border-gray-300 rounded-xl p-10

```

```

text-center hover:bg-gray-50 transition-colors relative">
    <input
        type="file"
        className="absolute inset-0 w-full h-full opacity-0 cursor-pointer"
        onChange={handleFileChange}
        disabled={status === 'uploading' || status === 'connecting'}
    />
    <UploadCloud className="w-12 h-12 text-blue-500 mx-auto mb-4" />
    <p className="text-gray-600 font-medium">
        {file ? file.name : 'Drag and drop a file or click to browse'}
    </p>
    {file && (
        <p className="text-sm text-gray-400 mt-2">
            {(file.size / 1024 / 1024).toFixed(2)} MB
        </p>
    )}
</div>

{ /* Прогресс-бар загрузки */ }
{file && status !== 'idle' && (
    <div className="mt-8">
        <div className="flex justify-between text-sm font-medium text-gray-
700 mb-2">
            <span>
                {status === 'connecting' ? 'Connecting...' :
                status === 'uploading' ? 'Uploading...' :
                status === 'success' ? 'Upload Complete' : 'Error'}
            </span>
            <span>{progress}%</span>
        </div>
        <div className="w-full bg-gray-200 rounded-full h-2.5 overflow-
hidden">
            <div
                className={`h-2.5 rounded-full transition-all duration-300 ${
                    status === 'error' ? 'bg-red-500' :
                    status === 'success' ? 'bg-green-500' : 'bg-blue-600'
                }`}
                style={{ width: `${progress}%` }}
            />
        </div>
    </div>
)}

{ /* Блок отображения ошибки */ }
{status === 'error' && (
    <div className="mt-4 p-4 bg-red-50 text-red-700 rounded-lg flex items-
start gap-3">
        <AlertCircle className="w-5 h-5 shrink-0 mt-0.5" />
        <p className="text-sm">{errorMsg}</p>
    </div>
)}

```

```

    { /* Блок подтверждения успешной загрузки */ }
    { status === 'success' && (
      <div className="mt-4 p-4 bg-green-50 text-green-700 rounded-lg flex
items-start gap-3">
        <CheckCircle className="w-5 h-5 shrink-0 mt-0.5" />
        <p className="text-sm">File uploaded successfully!</p>
      </div>
    ) }

    { /* Кнопка запуска загрузки */ }
    <div className="mt-8 flex justify-end">
      <button
        onClick={uploadFile}
        disabled={!file || status === 'uploading' || status === 'connecting'
|| status === 'success'}
        className="bg-blue-600 text-white px-6 py-2.5 rounded-lg font-medium
hover:bg-blue-700 transition-colors disabled:opacity-50 disabled:cursor-not-
allowed"
      >
        Start Upload
      </button>
    </div>
  </div>

  { /* Блок списка доступных файлов для скачивания */ }
  <div className="bg-white p-8 rounded-xl shadow-sm border border-gray-100">
    <div className="flex justify-between items-center mb-6">
      <h2 className="text-xl font-medium text-gray-900">Available Files</h2>
      { /* Кнопка ручного обновления списка */ }
      <button
        onClick={fetchFiles}
        disabled={isRefreshing}
        className={`p-2 text-gray-500 hover:text-blue-600 hover:bg-blue-50
rounded-full transition-colors ${
          isRefreshing ? 'animate-spin' : ''
        }`}
        title="Refresh List"
      >
        <RefreshCw className="w-5 h-5" />
      </button>
    </div>

    {availableFiles.length === 0 ? (
      <p className="text-gray-500 text-center py-8">No files available for
download.</p>
    ) : (
      <div className="space-y-3">
        {availableFiles.map((f) => (
          <div
            key={f.id}

```

```

        className="flex items-center justify-between p-4 border border-
gray-100 rounded-lg hover:bg-gray-50"
      >
        <div className="flex items-center gap-3">
          <div className="p-2 bg-blue-50 text-blue-600 rounded-lg">
            <File className="w-5 h-5" />
          </div>
          <div>
            <p className="font-medium text-gray-900">{f.filename}</p>
            <p className="text-xs text-gray-500">
              {(f.total_size / 1024 / 1024).toFixed(2)} MB •{' '}
              {new Date(f.end_time).toLocaleString()}
            </p>
            </* Прогресс-бар скачивания конкретного файла */>
            {downloadingFile === f.filename && (
              <div className="mt-2 w-48 bg-gray-200 rounded-full h-1.5
overflow-hidden">
                <div
                  className="bg-blue-600 h-1.5 rounded-full transition-
all duration-300"
                  style={{ width: `${downloadProgress}%` }}
                />
              </div>
            )}
          </div>
          </* Кнопка скачивания файла */>
          <button
            onClick={() => downloadFile(f.id)}
            className="p-2 text-gray-500 hover:text-blue-600 hover:bg-
blue-50 rounded-full transition-colors"
            title="Download"
            disabled={!downloadingFile} // Запрещаем новое скачивание
пока идёт текущее
          >
            {downloadingFile === f.filename ? (
              <span className="text-xs font-medium text-blue-600">
                {downloadProgress}%
              </span>
            ) : (
              <Download className="w-5 h-5" />
            )}
          </button>
        </div>
      </div>
    </div>
  </div>
);
}

```

```

// =====
// Login.tsx – Компонент экрана входа и регистрации.
// Реализует HTTP-аутентификацию пользователя, периодическую
// проверку доступности сервера и сохранение JWT-токена.
// =====

import React, { useState } from 'react';
import { Eye, EyeOff } from 'lucide-react';

// Компонент принимает колбэк onLogin, вызываемый при успешном входе.
// Передаёт объект пользователя родительскому компоненту App.
export default function Login({ onLogin }: { onLogin: (user: any) => void }) {

  // Флаг режима: false – вход, true – регистрация
  const [isRegister, setIsRegister] = useState(false);

  // Поля формы
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false); // Переключатель
видимости пароля
  const [role, setRole] = useState('user'); // Роль (только при
регистрации)

  // Состояние UI
  const [error, setError] = useState(''); // Сообщение об ошибке или
уведомление
  const [isLoading, setIsLoading] = useState(false); // Флаг выполнения запроса

  // Статус доступности сервера: 'checking' | 'ok' | 'error'
  const [serverStatus, setServerStatus] = useState<'checking' | 'ok' |
'error'>('checking');

  // =====
  // useEffect – периодическая проверка доступности сервера.
  // Выполняет GET /api/health каждые 5 секунд и обновляет
  // индикатор статуса в заголовке формы.
  // =====
  React.useEffect(() => {
    let isChecking = false; // Предотвращает одновременное выполнение нескольких
проверок

    const checkServer = async () => {
      if (isChecking) {
        console.log('Skipping checkServer, already checking');
        return;
      }
      isChecking = true;
      console.log('Checking server health...');

      try {

```

```

    // Устанавливаем таймаут 3 секунды для проверки доступности
    const controller = new AbortController();
    const timeoutId = setTimeout(() => {
      console.log('Aborting health check due to timeout');
      controller.abort();
    }, 3000);

    const res = await fetch('/api/health', { signal: controller.signal })
      .finally(() => clearTimeout(timeoutId));

    console.log('Health check response:', res.status);

    if (res.ok) {
      setServerStatus('ok');
      setError(''); // Сбрасываем сообщение об ошибке при восстановлении
сервера
    } else {
      setServerStatus('error');
      if (serverStatus !== 'error') {
        setError('Server is unreachable or database is locked.');
```

```

setIsLoading(true);
console.log('Form submitted', { isRegister, username, role });

// Выбираем эндпоинт и тело запроса в зависимости от режима
const endpoint = isRegister ? '/api/auth/register' : '/api/auth/login';
const payload = isRegister
  ? { username, password, role } // При регистрации передаём роль
  : { username, password }; // При входе роль не нужна

try {
  console.log('Sending request to', endpoint);

  // Таймаут 10 секунд на весь запрос аутентификации
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 10000);

  const res = await fetch(endpoint, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
    signal: controller.signal
  }).finally(() => clearTimeout(timeoutId));

  const data = await res.json();
  console.log('Response:', res.status, data);

  // При HTTP-ошибке выбрасываем исключение с сообщением от сервера
  if (!res.ok) {
    throw new Error(data.error || 'Something went wrong');
  }

  if (isRegister) {
    // Регистрация успешна — переключаем на форму входа
    setIsRegister(false);
    setError('Registration successful. Please login.');
```

сессии.

```

  } else {
    // Вход успешен — сохраняем токен и данные пользователя в хранилище
    // sessionStorage автоматически очищается при закрытии вкладки браузера.
    sessionStorage.setItem('token', data.token);
    sessionStorage.setItem('user', JSON.stringify(data.user));

    // Передаём объект пользователя в App для смены экрана
    onLogin(data.user);
  }
} catch (err: any) {
  console.error('Login error:', err);
  if (err.name === 'AbortError') {
    // Запрос превысил таймаут
    setError('Request timed out. Please try again.');
```

```

        setError(err.message);
    }
} finally {
    setIsLoading(false);
}
};

// =====
// Возвращаемая разметка компонента
// =====
return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50
dark:bg-gray-900 transition-colors duration-300">
        <div className="bg-white dark:bg-gray-800 p-8 rounded-2xl shadow-xl w-full
max-w-md border-t-4 border-google-blue dark:border-google-blue transition-colors
duration-300">

            {/* Заголовок с логотипом и индикатором статуса сервера */}
            <div className="text-center mb-8">
                <h1 className="text-3xl font-bold mb-2">
                    <span className="text-google-blue">G</span>
                    <span className="text-google-red">o</span>
                    <span className="text-google-yellow">o</span>
                    <span className="text-google-blue">g</span>
                    <span className="text-google-green">l</span>
                    <span className="text-google-red">e</span>
                    <span className="text-gray-700 dark:text-gray-200 ml-2 text-xl font-
normal">
                        FileTransfer
                    </span>
                </h1>
                <h2 className="text-xl font-medium text-gray-600 dark:text-gray-300">
                    {isRegister ? 'Create your account' : 'Sign in to continue'}
                </h2>
                {/* Индикатор доступности сервера: зелёный / жёлтый / красный */}
                <div className="mt-2 flex items-center justify-center gap-2 text-xs">
                    <span className={`w-2 h-2 rounded-full ${
                        serverStatus === 'ok' ? 'bg-green-500' :
                        serverStatus === 'checking' ? 'bg-yellow-500' : 'bg-red-500'
                    }`} />
                    <span className="text-gray-500 dark:text-gray-400">
                        {serverStatus === 'ok' ? 'Server Online' :
                        serverStatus === 'checking' ? 'Checking Server...' : 'Server
Offline'}
                    </span>
                </div>
            </div>

            {/* Блок сообщений об ошибках и уведомлений */}
            {error && (
                <div className={`p-3 mb-6 rounded-lg text-sm flex items-center gap-2 $

```

```

{
    error.includes('successful')
    ? 'bg-green-50 text-green-700 dark:bg-green-900/30 dark:text-
green-400 border border-green-200 dark:border-green-800'
    : 'bg-red-50 text-red-700 dark:bg-red-900/30 dark:text-red-400
border border-red-200 dark:border-red-800'
    `}>
    {error}
</div>
})

{ /* Форма входа / регистрации */ }
<form onSubmit={handleSubmit} className="space-y-5">

    { /* Поле имени пользователя */ }
    <div>
        <label className="block text-sm font-medium text-gray-700 dark:text-
gray-300 mb-1.5">
            Username
        </label>
        <input
            type="text"
            name="username"
            autoComplete="username"
            required
            className="w-full px-4 py-2.5 border border-gray-300 dark:border-
gray-600 rounded-lg focus:ring-2 focus:ring-google-blue focus:border-google-blue
outline-none transition-all bg-white dark:bg-gray-700 text-gray-900 dark:text-
white placeholder-gray-400 dark:placeholder-gray-500"
            value={username}
            onChange={e => setUsername(e.target.value)}
            placeholder="Enter your username"
        />
    </div>

    { /* Поле пароля с переключателем видимости */ }
    <div>
        <label className="block text-sm font-medium text-gray-700 dark:text-
gray-300 mb-1.5">
            Password
        </label>
        <div className="relative">
            <input
                type={showPassword ? 'text' : 'password'}
                name="password"
                autoComplete={isRegister ? 'new-password' : 'current-password'}
                required
                className="w-full px-4 py-2.5 border border-gray-300
dark:border-gray-600 rounded-lg focus:ring-2 focus:ring-google-blue
focus:border-google-blue outline-none transition-all bg-white dark:bg-gray-700
text-gray-900 dark:text-white placeholder-gray-400 dark:placeholder-gray-500 pr-

```

10"

```
        value={password}
        onChange={e => setPassword(e.target.value)}
        placeholder="Enter your password"
      />
      { /* Кнопка показать/скрыть пароль */ }
      <button
        type="button"
        onClick={() => setShowPassword(!showPassword)}
        className="absolute inset-y-0 right-0 px-3 flex items-center
text-gray-500 dark:text-gray-400 hover:text-gray-700 dark:hover:text-gray-200"
      >
        {showPassword ? <EyeOff className="w-5 h-5" /> : <Eye
className="w-5 h-5" />}
      </button>
    </div>
  </div>

  { /* Выбор роли — отображается только при регистрации */ }
  {isRegister && (
    <div>
      <label className="block text-sm font-medium text-gray-700
dark:text-gray-300 mb-1.5">
        Role
      </label>
      <div className="relative">
        <select
          className="w-full px-4 py-2.5 border border-gray-300
dark:border-gray-600 rounded-lg focus:ring-2 focus:ring-google-blue
focus:border-google-blue outline-none bg-white dark:bg-gray-700 text-gray-900
dark:text-white appearance-none"
          value={role}
          onChange={e => setRole(e.target.value)}
        >
          <option value="user">User</option>
          <option value="admin">Admin</option>
        </select>
        { /* Кастомная стрелка выпадающего списка */ }
        <div className="absolute inset-y-0 right-0 flex items-center px-
2 pointer-events-none">
          <svg className="w-4 h-4 text-gray-500 dark:text-gray-400"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M19 9l-7 7-7-7" />
          </svg>
        </div>
      </div>
    </div>
  )}

  { /* Кнопка отправки формы */ }
```

```

        <button
          type="submit"
          disabled={isLoading}
          className={`w-full py-2.5 rounded-lg text-white font-medium
transition-all transform active:scale-[0.98] shadow-md ${
            isRegister
              ? 'bg-google-green hover:bg-green-600 shadow-green-200
dark:shadow-none'
              : 'bg-google-blue hover:bg-blue-600 shadow-blue-200 dark:shadow-
none'
          } ${isLoading ? 'opacity-70 cursor-not-allowed' : ''}}`
        >
          {isLoading ? 'Processing...' : (isRegister ? 'Create Account' :
'Sign In')}
        </button>
      </form>

      {/* Ссылка переключения между входом и регистрацией */}
      <div className="mt-8 pt-6 border-t border-gray-100 dark:border-gray-700
text-center">
        <p className="text-sm text-gray-600 dark:text-gray-400 mb-3">
          {isRegister ? 'Already have an account?' : "Don't have an account?"}
        </p>
        <button
          onClick={() => { setIsRegister(!isRegister); setError(''); }}
          className="text-sm font-medium text-google-blue hover:underline"
        >
          {isRegister ? 'Sign in instead' : 'Create account'}
        </button>
      </div>
    </div>
  </div>
);
}

```

```

// =====
// AdminDashboard.tsx – Панель администратора программы-клиента.
// Предоставляет интерфейс для просмотра статистики сервера,
// управления журналом событий и мониторинга передач файлов.
// Доступна только пользователям с ролью 'admin'.
// =====

```

```

import { useState, useEffect } from 'react';
import {
  Activity, HardDrive, Users, Trash2, RefreshCw,
  FileText, Network, Edit2, X, Check, ChevronDown, ChevronUp
} from 'lucide-react';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
ResponsiveContainer } from 'recharts';

export default function AdminDashboard() {

```

```

// Активная вкладка панели: журнал, передачи или пакеты
const [activeTab, setActiveTab] = useState<'logs' | 'transfers' |
'packets'>('logs');

// --- Данные с сервера ---
const [stats, setStats] = useState<any>(null); // Статистика сервера
const [logs, setLogs] = useState<any[]>([]); // Журнал событий
const [transfers, setTransfers] = useState<any[]>([]); // Список передач
файлов
const [packets, setPackets] = useState<any[]>([]); // Список пакетов
const [connections, setConnections] = useState<any[]>([]); // Активные
подключения

const [loading, setLoading] = useState(true); // Флаг
первоначальной загрузки
const [error, setError] = useState<string | null>(null); // Сообщение об
ошибке загрузки

// --- Состояние редактирования записи журнала ---
const [editingLog, setEditingLog] = useState<any>(null); // Редактируемая
запись
const [editForm, setEditForm] = useState({ // Поля формы
  action: '', details: '', username: '', timestamp: ''
});

// --- Состояние развёрнутой передачи (детали + график) ---
const [expandedTransferId, setExpandedTransferId] = useState<number |
null>(null);
const [transferDetails, setTransferDetails] = useState<any>(null); // Детали
выбранной передачи

const [isRefreshing, setIsRefreshing] = useState(false); // Флаг обновления
данных

// --- Фильтры журнала событий ---
const [logFilterUser, setLogFilterUser] = useState(''); // Фильтр по
имени пользователя
const [logFilterAction, setLogFilterAction] = useState('all'); // Фильтр по
типу действия

// --- Состояние редактирования записи о передаче ---
const [editingTransfer, setEditingTransfer] = useState<any>(null);
const [editTransferForm, setEditTransferForm] = useState({ filename: '',
status: '' });

// =====
// handleEditTransfer – открывает форму редактирования передачи.
// Останавливает всплытие события чтобы не раскрыть/закрыть
// блок деталей передачи при нажатии кнопки редактирования.

```

```

// =====
const handleEditTransfer = (transfer: any, e: React.MouseEvent) => {
  e.stopPropagation();
  setEditingTransfer(transfer);
  setEditTransferForm({ filename: transfer.filename, status:
transfer.status });
};

// =====
// saveTransferEdit – сохраняет изменения записи о передаче.
// Отправляет PUT-запрос на /api/admin/transfers/:id и
// обновляет список передач локально без повторного запроса.
// =====
const saveTransferEdit = async (e: React.MouseEvent) => {
  e.stopPropagation();
  if (!editingTransfer) return;
  try {
    const res = await fetch(`/api/admin/transfers/${editingTransfer.id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(editTransferForm)
    });
    if (res.ok) {
      // Обновляем запись в локальном списке без перезагрузки всех данных
      setTransfers(transfers.map(t =>
        t.id === editingTransfer.id ? { ...t, ...editTransferForm } : t
      ));
      setEditingTransfer(null); // Закрываем форму редактирования
    }
  } catch (err) {
    console.error('Failed to update transfer', err);
  }
};

// =====
// deleteTransfer – удаляет запись о передаче и файл с сервера.
// Запрашивает подтверждение перед удалением.
// Отправляет DELETE-запрос на /api/admin/transfers/:id.
// =====
const deleteTransfer = async (id: number, e: React.MouseEvent) => {
  e.stopPropagation();
  if (!confirm('Are you sure you want to delete this transfer and its file?'))
return;
  try {
    const res = await fetch(`/api/admin/transfers/${id}`, { method:
'DELETE' });
    if (res.ok) {
      // Удаляем запись из локального списка
      setTransfers(transfers.filter(t => t.id !== id));
      // Закрываем детали если был раскрыт удалённый элемент
      if (expandedTransferId === id) setExpandedTransferId(null);
    }
  }
};

```

```

    }
  } catch (err) {
    console.error('Failed to delete transfer', err);
  }
};

// =====
// fetchData – загружает все данные панели администратора.
// Выполняет 5 параллельных запросов с таймаутом 5 секунд.
// Вызывается при монтировании компонента и каждые 5 секунд
// для автоматического обновления данных в реальном времени.
// =====
const fetchData = async () => {
  try {
    setIsRefreshing(true);
    setError(null);

    // Контроллер для отмены запросов при превышении таймаута
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 5000);

    // Выполняем все запросы параллельно для минимизации времени ожидания.
    // catch(e => null) предотвращает отказ всей группы при ошибке одного
    // запроса.
    const [statsRes, logsRes, transfersRes, packetsRes, connectionsRes] =
      await Promise.all([
        fetch('/api/admin/stats', { signal:
controller.signal }).catch(() => null),
        fetch('/api/admin/logs', { signal:
controller.signal }).catch(() => null),
        fetch('/api/admin/transfers', { signal:
controller.signal }).catch(() => null),
        fetch('/api/admin/packets', { signal:
controller.signal }).catch(() => null),
        fetch('/api/admin/connections', { signal:
controller.signal }).catch(() => null)
      ]);

    clearTimeout(timeoutId);

    // Обновляем состояние только при успешных ответах
    if (statsRes?.ok) setStats(await statsRes.json());
    if (logsRes?.ok) setLogs(await logsRes.json());
    if (transfersRes?.ok) setTransfers(await transfersRes.json());
    if (packetsRes?.ok) setPackets(await packetsRes.json());
    if (connectionsRes?.ok) setConnections(await connectionsRes.json());

  } catch (err: any) {
    console.error('Failed to fetch admin data', err);
    setError('Failed to load data.');
```

```

        setLoading(false);
        setIsRefreshing(false);
    }
};

// При монтировании компонента загружаем данные и запускаем
// автообновление с интервалом 5 секунд
useEffect(() => {
    fetchData();
    const interval = setInterval(fetchData, 5000);
    return () => clearInterval(interval); // Останавливаем интервал при
размонтировании
}, []);

// =====
// fetchTransferDetails – загружает детальную информацию
// о конкретной передаче: пакеты и метрики скорости.
// Повторный клик на ту же передачу скрывает детали.
// =====
const fetchTransferDetails = async (id: number) => {
    if (expandedTransferId === id) {
        setExpandedTransferId(null); // Скрываем если уже раскрыто
        return;
    }
    try {
        const res = await fetch(`/api/admin/transfers/${id}`);
        if (res.ok) {
            const data = await res.json();
            setTransferDetails(data);
            setExpandedTransferId(id);
        }
    } catch (err) {
        console.error('Failed to fetch transfer details', err);
    }
};

// =====
// handleEditLog – открывает форму редактирования записи журнала.
// Предзаполняет поля формы текущими значениями записи.
// =====
const handleEditLog = (log: any) => {
    setEditingLog(log);
    setEditForm({
        action: log.action,
        details: log.details,
        username: log.username,
        timestamp: log.timestamp
    });
};

// =====

```

```

// saveLogEdit – сохраняет изменения записи журнала.
// Отправляет PUT-запрос на /api/admin/logs/:id и обновляет
// запись в локальном списке без повторной загрузки всего журнала.
// =====
const saveLogEdit = async () => {
  if (!editingLog) return;
  try {
    const res = await fetch(`/api/admin/logs/${editingLog.id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(editForm)
    });
    if (res.ok) {
      setLogs(logs.map(l =>
        l.id === editingLog.id ? { ...l, ...editForm } : l
      ));
      setEditingLog(null); // Закрываем форму редактирования
    }
  } catch (err) {
    console.error('Failed to update log', err);
  }
};

// =====
// deleteLog – удаляет запись из журнала событий.
// Отправляет DELETE-запрос на /api/admin/logs/:id и удаляет
// запись из локального списка без перезагрузки журнала.
// =====
const deleteLog = async (id: number) => {
  try {
    const res = await fetch(`/api/admin/logs/${id}`, { method: 'DELETE' });
    if (res.ok) {
      setLogs(logs.filter(log => log.id !== id));
    }
  } catch (err) {
    console.error('Failed to delete log', err);
  }
};

// Показываем загрузку пока данные не загружены
if (loading && !stats) {
  return <div className="text-center py-12 text-gray-500">Loading
dashboard...</div>;
}

// =====
// Возвращаемая разметка компонента
// =====
return (
  <div className="space-y-6">

```

```

    {/ * Заголовок с кнопкой ручного обновления */}
    <div className="flex justify-between items-center">
      <h2 className="text-2xl font-medium text-gray-900">Admin Dashboard</h2>
      <div className="flex gap-3">
        <button
          onClick={fetchData}
          disabled={isRefreshing}
          className={`flex items-center gap-2 text-sm text-blue-600
            hover:text-blue-800 font-medium ${
              isRefreshing ? 'opacity-50 cursor-not-allowed' : ''
            }}`>
          >
          <RefreshCw className={`w-4 h-4 ${isRefreshing ? 'animate-spin' : ''}
            ${isRefreshing ? 'Refreshing...' : 'Refresh'}`>
        </button>
      </div>
    </div>

    {/ * Карточки статистики: подключения, файлы, данные */}
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6">

      {/ * Карточка активных подключений с всплывающим списком пользователей */}
      <div className="bg-white p-6 rounded-xl shadow-sm border border-gray-100
        flex items-center gap-4 relative group">
        <div className="p-3 bg-blue-50 text-blue-600 rounded-lg">
          <Activity className="w-6 h-6" />
        </div>
        <div>
          <p className="text-sm text-gray-500 font-medium">Active
            Connections</p>
          <p className="text-2xl font-semibold text-gray-900">
            {stats?.activeConnections || 0} / 3
          </p>
        </div>
        {/ * Всплывающий список подключённых пользователей при наведении */}
        <div className="absolute top-full left-0 mt-2 w-64 bg-white border
          border-gray-200 rounded-lg shadow-lg p-3 hidden group-hover:block z-10">
          <h4 className="text-xs font-semibold text-gray-500 uppercase mb-2">Connected Users</h4>
          {connections.length === 0 ? (
            <p className="text-xs text-gray-400">No active connections</p>
          ) : (
            <ul className="space-y-1">
              {connections.map((conn: any, i: number) => (
                <li key={i} className="text-xs flex justify-between">
                  <span className="font-medium text-gray-700">{conn.username}</span>
                  <span className="text-gray-400">
                    {new Date(conn.connectedAt).toLocaleTimeString()}

```

```

        </span>
      </li>
    )})
  </ul>
})
</div>
</div>

{ /* Карточка общего числа файлов */ }
<div className="bg-white p-6 rounded-xl shadow-sm border border-gray-100
flex items-center gap-4">
  <div className="p-3 bg-green-50 text-green-600 rounded-lg">
    <HardDrive className="w-6 h-6" />
  </div>
  <div>
    <p className="text-sm text-gray-500 font-medium">Total Files</p>
    <p className="text-2xl font-semibold text-gray-
900">{stats?.total_files || 0}</p>
  </div>
</div>

{ /* Карточка суммарного объёма переданных данных */ }
<div className="bg-white p-6 rounded-xl shadow-sm border border-gray-100
flex items-center gap-4">
  <div className="p-3 bg-purple-50 text-purple-600 rounded-lg">
    <Users className="w-6 h-6" />
  </div>
  <div>
    <p className="text-sm text-gray-500 font-medium">Total Data</p>
    <p className="text-2xl font-semibold text-gray-900">
      {((stats?.total_bytes || 0) / 1024 / 1024).toFixed(2)} MB
    </p>
  </div>
</div>
</div>

{ /* Навигационные вкладки */ }
<div className="border-b border-gray-200">
  <nav className="-mb-px flex gap-6">
    { /* Вкладка журнала событий */ }
    <button
      onClick={() => setActiveTab('logs')}
      className={`py-4 px-1 border-b-2 font-medium text-sm ${
        activeTab === 'logs'
          ? 'border-blue-500 text-blue-600'
          : 'border-transparent text-gray-500 hover:text-gray-700
hover:border-gray-300'
      }`}
    >
    <div className="flex items-center gap-2">
      <FileText className="w-4 h-4" /> System Logs

```

```

    </div>
  </button>
  { /* Вкладка передач файлов */ }
  <button
    onClick={() => setActiveTab('transfers')}
    className={`py-4 px-1 border-b-2 font-medium text-sm ${
      activeTab === 'transfers'
        ? 'border-blue-500 text-blue-600'
        : 'border-transparent text-gray-500 hover:text-gray-700
hover:border-gray-300'
    }}`
  >
    <div className="flex items-center gap-2">
      <Activity className="w-4 h-4" /> File Transfers
    </div>
  </button>
  { /* Вкладка инспектора пакетов */ }
  <button
    onClick={() => setActiveTab('packets')}
    className={`py-4 px-1 border-b-2 font-medium text-sm ${
      activeTab === 'packets'
        ? 'border-blue-500 text-blue-600'
        : 'border-transparent text-gray-500 hover:text-gray-700
hover:border-gray-300'
    }}`
  >
    <div className="flex items-center gap-2">
      <Network className="w-4 h-4" /> Packet Inspector
    </div>
  </button>
</nav>
</div>

{ /* =====
Вкладка «System Logs» – журнал событий с фильтрацией
и возможностью редактирования и удаления записей
===== */ }

{activeTab === 'logs' && (
  <div className="bg-white rounded-xl shadow-sm border border-gray-100
overflow-hidden">

    { /* Панель фильтров */ }
    <div className="p-4 border-b border-gray-100 flex gap-4 items-center
bg-gray-50">
      { /* Фильтр по имени пользователя */ }
      <div className="flex items-center gap-2">
        <span className="text-sm font-medium text-gray-600">Filter by
User:</span>
        <input
          type="text"
          placeholder="Username..."

```

```

        className="px-3 py-1.5 text-sm border border-gray-300 rounded-md
focus:outline-none focus:ring-1 focus:ring-blue-500"
        value={logFilterUser}
        onChange={(e) => setLogFilterUser(e.target.value)}
    />
</div>
{ /* Фильтр по типу действия */ }
<div className="flex items-center gap-2">
    <span className="text-sm font-medium text-gray-600">Filter by
Action:</span>
    <select
        className="px-3 py-1.5 text-sm border border-gray-300 rounded-md
focus:outline-none focus:ring-1 focus:ring-blue-500 bg-white"
        value={logFilterAction}
        onChange={(e) => setLogFilterAction(e.target.value)}
    >
        <option value="all">All Actions</option>
        <option value="login">Login</option>
        <option value="disconnect">Disconnect</option>
        <option value="upload_start">Upload Start</option>
        <option value="upload_end">Upload End</option>
    </select>
</div>
</div>

{ /* Таблица журнала событий */ }
<div className="overflow-x-auto">
    <table className="w-full text-sm text-left text-gray-500">
        <thead className="text-xs text-gray-700 uppercase bg-gray-50">
            <tr>
                <th className="px-6 py-3">Time</th>
                <th className="px-6 py-3">User</th>
                <th className="px-6 py-3">Action</th>
                <th className="px-6 py-3">Details</th>
                <th className="px-6 py-3 text-right">Manage</th>
            </tr>
        </thead>
        <tbody>
            { /* Применяем фильтры и отображаем строки журнала */ }
            {logs
                .filter(log => {
                    const matchesUser = logFilterUser
                    ? (log.username ||
''.toLowerCase()).includes(logFilterUser.toLowerCase())
                    : true;
                    const matchesAction = logFilterAction !== 'all'
                    ? log.action === logFilterAction
                    : true;
                    return matchesUser && matchesAction;
                })
                .map((log) => (

```

```
<tr key={log.id} className="bg-white border-b hover:bg-gray-  
50">  
  
    {/* Временная метка – редактируемая в режиме правки */}  
    <td className="px-6 py-4 whitespace-nowrap text-xs text-  
gray-500">  
  
        {editingLog?.id === log.id ? (  
            <input  
                value={editForm.timestamp}  
                onChange={e => setEditForm({ ...editForm, timestamp:  
e.target.value })}  
  
                className="border rounded px-2 py-1 w-full text-xs"  
            />  
            ) : new Date(log.timestamp).toLocaleString()}  
    </td>  
  
    {/* Имя пользователя */}  
    <td className="px-6 py-4 font-medium text-gray-900">  
        {editingLog?.id === log.id ? (  
            <input  
                value={editForm.username}  
                onChange={e => setEditForm({ ...editForm, username:  
e.target.value })}  
  
                className="border rounded px-2 py-1 w-full"  
            />  
            ) : (log.username || 'System')}  
    </td>  
  
    {/* Тип действия с цветовой маркировкой */}  
    <td className="px-6 py-4">  
        {editingLog?.id === log.id ? (  
            <input  
                value={editForm.action}  
                onChange={e => setEditForm({ ...editForm, action:  
e.target.value })}  
  
                className="border rounded px-2 py-1 w-full"  
            />  
            ) : (  
                <span className={`px-2.5 py-1 rounded-full text-xs  
font-medium border ${  
                    log.action === 'login'           ? 'bg-green-50 text-  
green-700 border-green-200' :  
                    log.action === 'disconnect'      ? 'bg-orange-50 text-  
orange-700 border-orange-200' :  
                    log.action === 'upload_start'     ? 'bg-blue-50 text-  
blue-700 border-blue-200' :  
                    log.action === 'upload_end'       ? 'bg-indigo-50 text-  
indigo-700 border-indigo-200' :  
                    'bg-gray-50 text-gray-700 border-gray-200'  
                }`}>  
                    {log.action === 'upload_start' ? 'Upload Started' :  
log.action === 'upload_end'   ? 'Upload  
Completed' :  
log.action.charAt(0).toUpperCase() +
```

```

log.action.slice(1)}
        </span>
      )}
    </td>
    { /* Описание события */ }
    <td className="px-6 py-4 text-gray-600">
      {editingLog?.id === log.id ? (
        <input
          value={editForm.details}
          onChange={e => setEditForm({ ...editForm, details:
e.target.value })}
          className="border rounded px-2 py-1 w-full"
        />
      ) : log.details}
    </td>
    { /* Кнопки управления записью */ }
    <td className="px-6 py-4 text-right flex justify-end gap-
2">
      {editingLog?.id === log.id ? (
        <>
          { /* Сохранить изменения */ }
          <button onClick={saveLogEdit} className="text-green-
600 hover:text-green-800">
            <Check className="w-4 h-4" />
          </button>
          { /* Отменить редактирование */ }
          <button onClick={() => setEditingLog(null)}
className="text-gray-500 hover:text-gray-700">
            <X className="w-4 h-4" />
          </button>
        </>
      ) : (
        <>
          { /* Открыть форму редактирования */ }
          <button onClick={() => handleEditLog(log)}
className="text-blue-600 hover:text-blue-800">
            <Edit2 className="w-4 h-4" />
          </button>
          { /* Удалить запись */ }
          <button onClick={() => deleteLog(log.id)}
className="text-red-500 hover:text-red-700">
            <Trash2 className="w-4 h-4" />
          </button>
        </>
      )}
    </td>
  </tr>
)}}
{logs.length === 0 && (
  <tr>
    <td colspan={5} className="px-6 py-8 text-center text-gray-

```

500">

```
                No logs found
            </td>
        </tr>
    })
</tbody>
</table>
</div>
</div>
})

{/* =====
    Вкладка «File Transfers» – список передач с возможностью
    раскрытия деталей: граф скорости и таблица пакетов
    ===== */}
{activeTab === 'transfers' && (
    <div className="space-y-4">
        {transfers.map((transfer) => (
            <div key={transfer.id} className="bg-white rounded-xl shadow-sm
border border-gray-100 overflow-hidden">

                {/* Заголовок передачи – кликабелен для раскрытия деталей */}
                <div
                    className="p-4 flex items-center justify-between cursor-pointer
hover:bg-gray-50"
                    onClick={() => fetchTransferDetails(transfer.id)}
                >
                    <div className="flex items-center gap-4">
                        <div className={`p-2 rounded-lg ${
                            transfer.status === 'completed' ? 'bg-green-100 text-green-
600' : 'bg-blue-100 text-blue-600'
                        }`}>
                            <FileText className="w-5 h-5" />
                        </div>
                        <div>
                            {/* Имя файла – редактируемое */}
                            {editingTransfer?.id === transfer.id ? (
                                <input
                                    value={editTransferForm.filename}
                                    onClick={(e) => e.stopPropagation()}
                                    onChange={(e) =>
setEditTransferForm({ ...editTransferForm, filename: e.target.value })}
                                    className="border rounded px-2 py-1 w-full text-sm font-
medium text-gray-900 mb-1"
                                />
                            ) : (
                                <p className="font-medium text-gray-
900">{transfer.filename}</p>
                            )}
                            <p className="text-xs text-gray-500">
                                {(transfer.total_size / 1024 / 1024).toFixed(2)} MB •{' '}
```

```

        {new Date(transfer.start_time).toLocaleString()}
      </p>
    </div>
  </div>

  <div className="flex items-center gap-4">
    {editingTransfer?.id === transfer.id ? (
      /* Форма редактирования статуса передачи */
      <div className="flex items-center gap-2" onClick={(e) =>
e.stopPropagation()}>
        <select
          value={editTransferForm.status}
          onChange={(e) =>
setEditTransferForm({ ...editTransferForm, status: e.target.value })}
          className="text-xs border rounded px-2 py-1"
        >
          <option value="uploading">Uploading</option>
          <option value="completed">Completed</option>
          <option value="error">Error</option>
        </select>
        <button onClick={saveTransferEdit} className="text-green-
600 hover:text-green-800">
          <Check className="w-4 h-4" />
        </button>
        <button
          onClick={(e) => { e.stopPropagation();
setEditingTransfer(null); }}
          className="text-gray-500 hover:text-gray-700"
        >
          <X className="w-4 h-4" />
        </button>
      </div>
    ) : (
      <>
        {/* Бейдж статуса передачи */}
        <span className={`px-2.5 py-1 rounded-full text-xs font-
medium ${
          transfer.status === 'completed' ? 'bg-green-100 text-
green-800' : 'bg-blue-100 text-blue-800'
        }`}>
          {transfer.status}
        </span>
        <div className="flex items-center gap-2 ml-2">
          <button
            onClick={(e) => handleEditTransfer(transfer, e)}
            className="text-blue-600 hover:text-blue-800 p-1
rounded hover:bg-blue-50"
          >
            <Edit2 className="w-4 h-4" />
          </button>
          <button

```

```

        onClick={e => deleteTransfer(transfer.id, e)}
        className="text-red-500 hover:text-red-700 p-1 rounded
hover:bg-red-50"
    >
        <Trash2 className="w-4 h-4" />
    </button>
</div>
{/* Иконка раскрытия/скрытия деталей */}
{expandedTransferId === transfer.id
    ? <ChevronUp className="w-5 h-5 text-gray-400" />
    : <ChevronDown className="w-5 h-5 text-gray-400" />}
</>
    )}
</div>
</div>

{/* Детальная информация о передаче: график и пакеты */}
{expandedTransferId === transfer.id && transferDetails && (
    <div className="border-t border-gray-100 p-6 bg-gray-50">

        {/* График скорости передачи данных */}
        <h4 className="text-sm font-medium text-gray-700 mb-
4">Transfer Speed (Bytes/sec)</h4>
        <div className="h-64 bg-white rounded-lg p-4 border border-
gray-200 mb-6">
            <ResponsiveContainer width="100%" height="100%" minWidth={0}
minHeight={0}>
                <LineChart data={transferDetails.metrics}>
                    <CartesianGrid strokeDasharray="3 3" />
                    <XAxis
                        dataKey="timestamp"
                        tickFormatter={(t) => new
Date(t).toLocaleTimeString()}
                    />
                    <YAxis />
                    <Tooltip labelFormatter={(t) => new
Date(t).toLocaleString()} />
                    {/* Линия скорости в байтах/сек */}
                    <Line
                        type="monotone"
                        dataKey="speed_bps"
                        stroke="#2563eb"
                        strokeWidth={2}
                        dot={false}
                    />
                </LineChart>
            </ResponsiveContainer>
        </div>

        {/* Таблица захваченных пакетов */}
        <h4 className="text-sm font-medium text-gray-700 mb-4">Packet

```

Capture</h4>

```
<div className="bg-white rounded-lg border border-gray-200
overflow-hidden">
  <table className="w-full text-xs text-left text-gray-500">
    <thead className="bg-gray-100 text-gray-700 uppercase">
      <tr>
        <th className="px-4 py-2">Time</th>
        <th className="px-4 py-2">Dir</th>
        <th className="px-4 py-2">Type</th>
        <th className="px-4 py-2">Size</th>
        <th className="px-4 py-2">Payload (Hex)</th>
      </tr>
    </thead>
    <tbody>
      {transferDetails.packets.map((pkt: any) => (
        <tr key={pkt.id} className="border-b hover:bg-gray-50
font-mono">
          <td className="px-4 py-2">
            {new Date(pkt.timestamp).toLocaleTimeString()}
          </td>
          <td className="px-4 py-2">
            {/* Цветовая маркировка направления: зелёный –
входящий, синий – исходящий */}
            <span className={pkt.direction === 'in' ? 'text-
green-600' : 'text-blue-600'}>
              {pkt.direction === 'in' ? '← IN' : '→ OUT'}
            </span>
          </td>
          <td className="px-4 py-2">{pkt.type}</td>
          <td className="px-4 py-2">{pkt.size} B</td>
          <td className="px-4 py-2 text-gray-400 truncate max-
w-xs">
            {pkt.payload_preview}
          </td>
        </tr>
      )})}
    </tbody>
  </table>
</div>
</div>
  )}
</div>
  )})}
</div>
)}

{/* =====
Вкладка «Packet Inspector» – сводная таблица всех пакетов
последних передач в хронологическом порядке
===== */}
{activeTab === 'packets' && (
```

```

    <div className="bg-white rounded-xl shadow-sm border border-gray-100
overflow-hidden">
      <div className="overflow-x-auto">
        <table className="w-full text-xs text-left text-gray-500 font-mono">
          <thead className="bg-gray-50 text-gray-700 uppercase">
            <tr>
              <th className="px-4 py-3">Time</th>
              <th className="px-4 py-3">Direction</th>
              <th className="px-4 py-3">Type</th>
              <th className="px-4 py-3">Size</th>
              <th className="px-4 py-3">Payload Preview</th>
            </tr>
          </thead>
          <tbody>
            {packets.map((pkt) => (
              <tr key={pkt.id} className="border-b hover:bg-gray-50">
                <td className="px-4 py-2 whitespace-nowrap">
                  {new Date(pkt.timestamp).toLocaleTimeString()}
                </td>
                <td className="px-4 py-2">
                  {/* RX – принятый пакет, TX – отправленный */}
                  <span className={`px-2 py-0.5 rounded text-xs font-bold ${
                    pkt.direction === 'in'
                      ? 'bg-green-100 text-green-700'
                      : 'bg-blue-100 text-blue-700'
                  }`}>
                    {pkt.direction === 'in' ? 'RX' : 'TX'}
                  </span>
                </td>
                <td className="px-4 py-2 font-semibold text-gray-
700">{pkt.type}</td>
                <td className="px-4 py-2">{pkt.size}</td>
                {/* Hex-предпросмотр первых байт полезной нагрузки */}
                <td className="px-4 py-2 text-gray-400 truncate max-w-md">
                  {pkt.payload_preview}
                </td>
              </tr>
            )})}
          </tbody>
        </table>
      </div>
    </div>
  )}
</div>
);
}

```

```

// =====
// App.tsx – Корневой компонент программы-клиента.
// Отвечает за маршрутизацию между экранами: в зависимости
// от роли аутентифицированного пользователя отображает

```

```

// UserDashboard или AdminDashboard. Управляет тёмной темой.
// =====

import { useState, useEffect } from 'react';
import Login from '../components/Login';
import UserDashboard from '../components/UserDashboard';
import AdminDashboard from '../components/AdminDashboard';
import { Moon, Sun } from 'lucide-react';

export default function App() {

  // Объект текущего пользователя. null – пользователь не вошёл.
  const [user, setUser] = useState<any>(null);

  // Флаг тёмной темы интерфейса
  const [darkMode, setDarkMode] = useState(false);

  // =====
  // useEffect – инициализация при первом запуске.
  // Восстанавливает сессию пользователя из sessionStorage
  // и применяет сохранённые настройки темы оформления.
  // =====
  useEffect(() => {
    try {
      // Пытаемся восстановить сессию из хранилища браузера.
      // Токен и данные пользователя были сохранены при входе в Login.tsx.
      const storedUser = sessionStorage.getItem('user');
      if (storedUser) {
        setUser(JSON.parse(storedUser));
      }
    } catch (e) {
      // Если данные повреждены – очищаем хранилище и требуем повторного входа
      console.error('Failed to parse user from session storage', e);
      sessionStorage.removeItem('user');
      sessionStorage.removeItem('token');
    }

    // Определяем тему: сначала проверяем сохранённое предпочтение,
    // затем системные настройки операционной системы пользователя
    const storedTheme = localStorage.getItem('theme');
    if (storedTheme === 'dark' ||
      (!storedTheme && window.matchMedia('(prefers-color-scheme:
dark)').matches)) {
      setDarkMode(true);
      document.documentElement.classList.add('dark'); // Применяем тёмную тему
      // через CSS-класс
    }
  }, []);

  // =====
  // toggleDarkMode – переключает тёмную/светлую тему.

```

```

// Сохраняет выбор в localStorage для восстановления при
// следующем открытии приложения.
// =====
const toggleDarkMode = () => {
  const newMode = !darkMode;
  setDarkMode(newMode);
  if (newMode) {
    document.documentElement.classList.add('dark');
    localStorage.setItem('theme', 'dark');
  } else {
    document.documentElement.classList.remove('dark');
    localStorage.setItem('theme', 'light');
  }
};

// =====
// handleLogout – завершает сессию пользователя.
// Удаляет JWT-токен и данные пользователя из sessionStorage
// и сбрасывает состояние – приложение возвращается к экрану входа.
// =====
const handleLogout = () => {
  sessionStorage.removeItem('token');
  sessionStorage.removeItem('user');
  setUser(null); // Сброс state вызывает перерисовку с отображением Login
};

// =====
// Если пользователь не вошёл – отображаем экран входа.
// Кнопка переключения темы доступна и на экране входа.
// =====
if (!user) {
  return (
    <div className={darkMode ? 'dark' : ''}>
      {/* Кнопка переключения темы в правом верхнем углу */}
      <div className="absolute top-4 right-4 z-50">
        <button
          onClick={toggleDarkMode}
          className="p-2 rounded-full bg-gray-100 dark:bg-gray-800 text-gray-
800 dark:text-gray-200 hover:bg-gray-200 dark:hover:bg-gray-700 transition-
colors"
        >
          {darkMode ? <Sun className="w-5 h-5" /> : <Moon className="w-5 h-
5" />}
        </button>
      </div>
      {/* Компонент Login вызывает setUser через колбэк onLogin при успешном
входе */}
      <Login onLogin={setUser} />
    </div>
  );
}

```

```

// =====
// Основной интерфейс для аутентифицированного пользователя.
// Маршрутизация по роли:
//   'admin' → AdminDashboard (управление журналом и статистика)
//   'user' → UserDashboard (загрузка и скачивание файлов)
// =====
return (
  <div className={`min-h-screen font-sans transition-colors duration-300 ${
    darkMode ? 'dark bg-gray-900 text-gray-100' : 'bg-gray-50 text-gray-900'
  }`} >

    {/* Верхняя панель навигации */}
    <header className="bg-white dark:bg-gray-800 shadow-sm px-6 py-4 flex
justify-between items-center transition-colors duration-300">
      <h1 className="text-xl font-medium text-google-blue dark:text-google-
blue">
        FileTransfer
      </h1>
      <div className="flex items-center gap-4">
        {/* Кнопка переключения тёмной/светлой темы */}
        <button
          onClick={toggleDarkMode}
          className="p-2 rounded-full bg-gray-100 dark:bg-gray-700 text-gray-
600 dark:text-gray-300 hover:bg-gray-200 dark:hover:bg-gray-600 transition-
colors"
        >
          {darkMode ? <Sun className="w-4 h-4" /> : <Moon className="w-4 h-
4" />}
        </button>

        {/* Информация о текущем пользователе */}
        <span className="text-sm text-gray-600 dark:text-gray-300">
          Logged in as <b>{user.username}</b> ({user.role})
        </span>

        {/* Кнопка выхода из системы */}
        <button
          onClick={handleLogout}
          className="text-sm text-google-red hover:text-red-700 font-medium"
        >
          Logout
        </button>
      </div>
    </header>

    {/* Основная область контента */}
    <main className="p-6 max-w-5xl mx-auto">
      {/* Выбор компонента по роли пользователя */}
      {user.role === 'admin'
        ? <AdminDashboard />           // Панель администратора

```

```
        : <UserDashboard user={user} /> // Панель обычного пользователя
    }
    </main>
</div>
);
}
```

Приложение В

(обязательное)

Код программы-сервера

```
// =====  
  
// server.ts – Главный файл программы-сервера.  
// Реализует HTTP REST API, WebSocket-сервер протокола  
// прикладного уровня, аутентификацию пользователей,  
// приём и передачу файлов, протоколирование операций.  
// =====  
  
import express from 'express';  
import { createServer as createViteServer } from 'vite';  
import { WebSocketServer, WebSocket } from 'ws';  
import { Worker } from 'worker_threads';  
import path from 'path';  
import bcrypt from 'bcryptjs';           // Библиотека хэширования паролей  
import jwt from 'jsonwebtoken';           // Библиотека работы с JWT-токенами  
import db from './src/db.js';             // Модуль базы данных SQLite  
import archiver from 'archiver';          // Библиотека создания ZIP-архивов  
import fs from 'fs';                       // Стандартный модуль файловой системы  
Node.js  
  
// Секретный ключ для подписи JWT-токенов.  
// В рабочей среде должен задаваться через переменную окружения.  
const JWT_SECRET = process.env.JWT_SECRET || 'super_secret_key_123';  
  
// Порт, на котором запускается HTTP-сервер  
const PORT = 3000;  
  
// =====  
// startServer – асинхронная функция запуска сервера.  
// Инициализирует Express, WebSocket-сервер, подключает  
// маршруты REST API, запускает Worker Thread и heartbeat.  
// =====  
async function startServer() {  
  const app = express();  
  
  // Middleware логирования: выводит метод и URL каждого входящего запроса  
  app.use((req, res, next) => {  
    console.log(`[Server] ${req.method} ${req.url}`);  
    next();  
  });  
  
  // Middleware парсинга JSON-тела запроса  
  app.use(express.json());  
  
  // Запускаем HTTP-сервер на всех сетевых интерфейсах (0.0.0.0)  
  const server = app.listen(PORT, '0.0.0.0', () => {
```

```

    console.log(`Server running on http://localhost:${PORT}`);
  });

  // -----
  // Создаём WebSocket-сервер в режиме noServer.
  // Это позволяет вручную управлять HTTP Upgrade-запросами,
  // разделяя WebSocket-трафик и трафик Vite HMR.
  // -----
  const wss = new WebSocketServer({ noServer: true });

  // Перехватываем HTTP Upgrade-запросы для переключения на WebSocket.
  // Только запросы на /api/ws передаются WebSocket-серверу;
  // остальные (Vite HMR) обрабатываются Vite самостоятельно.
  server.on('upgrade', (request, socket, head) => {
    if (request.url === '/api/ws') {
      wss.handleUpgrade(request, socket, head, (ws) => {
        wss.emit('connection', ws, request);
      });
    } else {
      return; // Передаём Vite для обработки Hot Module Replacement
    }
  });

  // =====
  // REST API маршруты
  // =====

  // -----
  // GET /api/health – проверка доступности сервера и базы данных.
  // Вызывается программой-клиентом периодически для отображения
  // индикатора статуса сервера на экране входа.
  // -----
  app.get('/api/health', (req, res) => {
    try {
      db.prepare('SELECT 1').get(); // Минимальный запрос для проверки БД
      res.json({ status: 'ok', db: 'connected' });
    } catch (err: any) {
      console.error('[Server] Health check failed:', err);
      res.status(500).json({ status: 'error', message: err.message });
    }
  });

  // -----
  // GET /api/admin/transfers – список всех передач файлов.
  // Возвращает последние 50 записей, отсортированных по времени.
  // Доступен только администратору через панель AdminDashboard.
  // -----
  app.get('/api/admin/transfers', (req, res) => {
    const transfers = db.prepare(
      'SELECT * FROM transfers ORDER BY start_time DESC LIMIT 50'
    ).all();
  });

```

```

    res.json(transfers);
  });

  // -----
  // DELETE /api/admin/transfers/:id – удаление записи о передаче.
  // Удаляет файл с диска, связанные пакеты, метрики и саму запись.
  // -----
  app.delete('/api/admin/transfers/:id', (req, res) => {
    const transfer = db.prepare(
      'SELECT * FROM transfers WHERE id = ?'
    ).get(req.params.id) as any;

    if (!transfer) return res.status(404).json({ error: 'Transfer not found' });

    try {
      // Удаляем физический файл с диска если он существует
      if (transfer.file_path && fs.existsSync(transfer.file_path)) {
        fs.unlinkSync(transfer.file_path);
      }

      // Удаляем связанные записи из дочерних таблиц
      db.prepare('DELETE FROM packets WHERE transfer_id
= ?').run(req.params.id);
      db.prepare('DELETE FROM transfer_metrics WHERE transfer_id
= ?').run(req.params.id);
      // Удаляем саму запись о передаче
      db.prepare('DELETE FROM transfers WHERE id = ?').run(req.params.id);

      res.json({ success: true });
    } catch (err: any) {
      console.error('Failed to delete transfer', err);
      res.status(500).json({ error: 'Failed to delete transfer' });
    }
  });

  // -----
  // PUT /api/admin/transfers/:id – редактирование записи о передаче.
  // Позволяет администратору изменить имя файла и статус передачи.
  // -----
  app.put('/api/admin/transfers/:id', (req, res) => {
    const { filename, status } = req.body;
    db.prepare(
      'UPDATE transfers SET filename = ?, status = ? WHERE id = ?'
    ).run(filename, status, req.params.id);
    res.json({ success: true });
  });

  // -----
  // GET /api/files – список файлов, доступных для скачивания.
  // Возвращает только завершённые передачи с существующим файлом.
  // Используется компонентом UserDashboard для отображения списка.

```

```

// -----
app.get('/api/files', (req, res) => {
  const files = db.prepare(`
    SELECT id, filename, total_size, end_time, user_id
    FROM transfers
    WHERE status = "completed" AND file_path IS NOT NULL
    ORDER BY end_time DESC
  `).all();
  res.json(files);
});

// -----
// GET /api/files/:id/download – скачивание файла по HTTP.
// Резервный маршрут для прямого HTTP-скачивания (не через WebSocket).
// -----
app.get('/api/files/:id/download', (req, res) => {
  const transfer = db.prepare(
    'SELECT * FROM transfers WHERE id = ?'
  ).get(req.params.id) as any;

  if (!transfer || !transfer.file_path) {
    return res.status(404).json({ error: 'File not found' });
  }
  if (!fs.existsSync(transfer.file_path)) {
    return res.status(404).json({ error: 'File not found on disk' });
  }

  // Отправляем файл с оригинальным именем через HTTP
  res.download(transfer.file_path, transfer.filename);
});

// -----
// GET /api/admin/transfers/:id – детальная информация о передаче.
// Возвращает запись о передаче, последние 200 пакетов и
// последние 1000 точек метрик скорости для построения графика.
// -----
app.get('/api/admin/transfers/:id', (req, res) => {
  const transfer = db.prepare(
    'SELECT * FROM transfers WHERE id = ?'
  ).get(req.params.id);

  if (!transfer) return res.status(404).json({ error: 'Transfer not found' });

  // Ограничиваем число пакетов чтобы не вызвать зависание браузера
  const packets = db.prepare(`
    SELECT * FROM packets
    WHERE transfer_id = ?
    ORDER BY timestamp DESC LIMIT 200
  `).all(req.params.id);

  // Ограничиваем число точек метрик для производительности

```

```

const metrics = db.prepare(`
  SELECT * FROM transfer_metrics
  WHERE transfer_id = ?
  ORDER BY timestamp DESC LIMIT 1000
`).all(req.params.id);

// Разворачиваем в хронологический порядок
res.json({ transfer, packets: packets.reverse(), metrics:
metrics.reverse() });
});

// -----
// GET /api/admin/packets – последние 200 пакетов всех передач.
// Используется вкладкой «Packet Inspector» в AdminDashboard.
// -----
app.get('/api/admin/packets', (req, res) => {
  const packets = db.prepare(
    'SELECT * FROM packets ORDER BY timestamp DESC LIMIT 200'
  ).all();
  res.json(packets);
});

// -----
// GET /api/download-source – скачивание исходного кода проекта.
// Формирует ZIP-архив исходников и отправляет его клиенту.
// Исключает node_modules, dist, uploads, .git и базу данных.
// -----
app.get('/api/download-source', (req, res) => {
  console.log('[Server] Download source request received');
  try {
    // Создаём ZIP-архив с максимальным уровнем сжатия (9)
    const archive = archiver('zip', { zlib: { level: 9 } });

    res.attachment('project-source.zip');

    archive.on('error', (err) => {
      console.error('[Server] Archiver error:', err);
      if (!res.headersSent) res.status(500).send({ error: err.message });
    });

    archive.on('warning', (err) => {
      if (err.code === 'ENOENT') {
        console.warn('[Server] Archiver warning:', err);
      } else {
        console.error('[Server] Archiver error:', err);
        if (!res.headersSent) res.status(500).send({ error: err.message });
      }
    });
  }
});

// Направляем поток архива напрямую в HTTP-ответ
archive.pipe(res);

```

```

// Добавляем все файлы проекта, исключая служебные директории
archive.glob('**/*', {
  cwd: process.cwd(),
  ignore: ['node_modules/**', 'dist/**', 'uploads/**', '.git/**',
    'data.db', 'data.db-journal', '.env']
});

archive.finalize();
} catch (err) {
  console.error('[Server] Download source unexpected error:', err);
  if (!res.headersSent) {
    res.status(500).send({ error: 'Internal server error during
download' });
  }
}
});

// -----
// POST /api/auth/register – регистрация нового пользователя.
// Хэширует пароль с коэффициентом сложности 10 и сохраняет
// пользователя в базе данных. При дублировании имени
// возвращает ошибку 400.
// -----
app.post('/api/auth/register', async (req, res) => {
  console.log('[Server] Register request received:', req.body.username);
  const { username, password, role } = req.body;
  try {
    const hash = await bcrypt.hash(password, 10); // Хэширование пароля
    const stmt = db.prepare(
      'INSERT INTO users (username, password_hash, role) VALUES (?, ?, ?)'
    );
    const info = stmt.run(username, hash, role || 'user');
    console.log('[Server] Register success:', username);
    res.json({ success: true, userId: info.lastInsertRowid });
  } catch (err: any) {
    console.error('[Server] Register error:', err.message);
    if (err.message.includes('UNIQUE constraint failed')) {
      // Имя пользователя уже занято
      res.status(400).json({ error: 'Username already exists' });
    } else {
      res.status(500).json({ error: 'Internal server error' });
    }
  }
});

// -----
// POST /api/auth/login – аутентификация пользователя.
// Проверяет учётные данные, генерирует JWT-токен со сроком
// действия 24 часа и записывает событие входа в журнал.
// -----

```

```

app.post('/api/auth/login', async (req, res) => {
  console.log('[Server] Login request received:', req.body.username);
  const { username, password } = req.body;
  try {
    // Ищем пользователя в базе данных по имени
    const user = db.prepare(
      'SELECT * FROM users WHERE username = ?'
    ).get(username) as any;

    if (!user) {
      console.log('[Server] Login failed: User not found');
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    // Сравниваем введенный пароль с сохранённым хэшем
    const validPassword = await bcrypt.compare(password, user.password_hash);
    if (!validPassword) {
      console.log('[Server] Login failed: Invalid password');
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    // Формируем JWT-токен с полезной нагрузкой: id, username, role
    const token = jwt.sign(
      { id: user.id, username: user.username, role: user.role },
      JWT_SECRET,
      { expiresIn: '24h' }
    );

    // Записываем событие входа в журнал
    db.prepare(
      'INSERT INTO logs (user_id, username, action, details) VALUES
      (?, ?, ?, ?)'
    ).run(user.id, user.username, 'login', 'User logged in');

    console.log('[Server] Login success:', username);
    res.json({ token, user: { id: user.id, username: user.username, role:
user.role } });
  } catch (error) {
    console.error('[Server] Login error:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
});

// -----
// GET /api/admin/stats – статистика сервера.
// Возвращает общее число файлов, суммарный объём данных
// и текущее число активных WebSocket-подключений.
// -----
app.get('/api/admin/stats', (req, res) => {
  const stats = db.prepare('SELECT * FROM stats WHERE id = 1').get();
  const activeConnections = wss.clients.size; // Число активных WS-клиентов

```

```

    res.json({ ...stats, activeConnections });
  });

  // -----
  // GET /api/admin/logs – журнал событий сервера.
  // Возвращает последние 100 записей в обратном хронологическом порядке.
  // -----
  app.get('/api/admin/logs', (req, res) => {
    const logs = db.prepare(`
      SELECT id, username, action, details, timestamp
      FROM logs
      ORDER BY timestamp DESC
      LIMIT 100
    `).all();
    res.json(logs);
  });

  // -----
  // DELETE /api/admin/logs/:id – удаление записи журнала.
  // Доступно только администратору.
  // -----
  app.delete('/api/admin/logs/:id', (req, res) => {
    db.prepare('DELETE FROM logs WHERE id = ?').run(req.params.id);
    res.json({ success: true });
  });

  // -----
  // PUT /api/admin/logs/:id – редактирование записи журнала.
  // Позволяет администратору изменить действие, описание,
  // имя пользователя и временную метку записи.
  // -----
  app.put('/api/admin/logs/:id', (req, res) => {
    const { action, details, username, timestamp } = req.body;
    db.prepare(`
      UPDATE logs
      SET action = ?, details = ?, username = ?, timestamp = ?
      WHERE id = ?
    `).run(action, details, username, timestamp, req.params.id);
    res.json({ success: true });
  });

  // -----
  // Подключаем Vite как middleware в режиме разработки.
  // Vite обслуживает клиентское приложение и обеспечивает
  // Hot Module Replacement для мгновенного обновления интерфейса.
  // В production-режиме отдаётся собранная статика из папки dist.
  // -----
  if (process.env.NODE_ENV !== 'production') {
    const vite = await createViteServer({
      server: {
        middlewareMode: true,

```

```

    hmr: { server } // Привязываем HMR к нашему HTTP-серверу
  },
  appType: 'spa', // Single Page Application
});
app.use(vite.middlewares);
} else {
  app.use(express.static('dist'));
}

// -----
// Worker Thread для записи файлов на диск.
// Вынесен в отдельный поток чтобы не блокировать основной
// цикл событий Node.js при интенсивных файловых операциях.
// -----
const fileWorker = new Worker(path.join(process.cwd(), 'src', 'worker.js'));

fileWorker.on('error', (err) => {
  console.error('Worker error:', err);
});

// Обрабатываем сообщения от Worker Thread:
// - ack_start: воркер открыл файл и сообщает путь к нему
// - ack_end: воркер закрыл файл, увеличиваем счётчик файлов
fileWorker.on('message', (msg) => {
  if (msg.type === 'ack_start') {
    // Сохраняем путь к файлу в базе данных для последующего скачивания
    db.prepare(
      'UPDATE transfers SET file_path = ? WHERE file_id = ?'
    ).run(msg.filePath, msg.fileId.toString());
  } else if (msg.type === 'ack_end') {
    // Увеличиваем счётчик успешно сохранённых файлов
    db.prepare(
      'UPDATE stats SET total_files = total_files + 1 WHERE id = 1'
    ).run();
  }
});

// =====
// WebSocket-сервер: обработка подключений и сообщений протокола
// =====

wss.on('connection', (ws: WebSocket, req) => {

  // Проверяем лимит одновременных подключений.
  // При превышении максимума (3 клиента) закрываем соединение
  // с кодом 1008 (Policy Violation) и сообщением об ошибке.
  if (wss.clients.size > 3) {
    ws.close(1008, 'Server full (max 3 clients)');
    return;
  }
}

```

```

    // Переменные состояния текущего соединения
    let authenticatedUser: any = null;    // Данные аутентифицированного
пользователя
    let currentFileId: number | null = null;    // Идентификатор текущей
сессии загрузки
    let dbTransferId: number | null = null;    // ID записи о передаче в
базе данных
    let currentBytesReceived = 0;    // Счётчик принятых байт
текущего файла
    let lastMetricTime = Date.now();    // Время последнего
сохранения метрики
    let bytesSinceLastMetric = 0;    // Байт принято с последней
метрики

    // Буфер для пакетной записи пакетов в БД.
    // Прямая запись каждого пакета отдельно вызывает блокировку БД
    // при высокой скорости передачи, поэтому используется буферизация.
    let packetLogBuffer: any[] = [];
    let packetLogTimeout: NodeJS.Timeout | null = null;

    // -----
    // flushPacketLogs – сбрасывает буфер пакетов в базу данных.
    // Использует транзакцию для атомарной вставки всех накопленных
    // записей за одну операцию, что значительно быстрее
    // последовательных одиночных INSERT-запросов.
    // -----
    const flushPacketLogs = () => {
        if (packetLogBuffer.length === 0) return;

        try {
            const bufferToInsert = [...packetLogBuffer];
            packetLogBuffer = []; // Очищаем буфер перед записью

            const insert = db.prepare(
                'INSERT INTO packets (transfer_id, direction, type, size,
payload_preview) VALUES (?, ?, ?, ?, ?)'
            );
            // Оборачиваем вставку в транзакцию для производительности
            const insertMany = db.transaction((packets) => {
                for (const pkt of packets) {
                    insert.run(pkt.transferId, pkt.direction, pkt.type, pkt.size,
pkt.preview);
                }
            });
            insertMany(bufferToInsert);
        } catch (e) {
            console.error('Packet log flush error', e);
        }
    };

    // -----

```

```

// logPacket – добавляет запись о пакете в буфер.
// Для бинарных данных сохраняет первые 20 байт в hex-формате,
// для текстовых – первые 50 символов.
// Автоматически сбрасывает буфер при накоплении 100 записей
// или через 1 секунду после последней добавленной записи.
// -----
const logPacket = (
  direction: 'in' | 'out',
  type: string,
  size: number,
  payload: Buffer | string
) => {
  try {
    // Формируем предпросмотр содержимого пакета
    const preview = Buffer.isBuffer(payload)
      ? payload.subarray(0, 20).toString('hex') // Бинарные данные → hex
      : payload.toString().substring(0, 50);    // Текст → первые 50
СИМВОЛОВ

    packetLogBuffer.push({ transferId: dbTransferId, direction, type, size,
    preview });

    if (packetLogBuffer.length >= 100) {
      // Буфер заполнен – немедленно сбрасываем в БД
      if (packetLogTimeout) clearTimeout(packetLogTimeout);
      packetLogTimeout = null;
      flushPacketLogs();
    } else if (!packetLogTimeout) {
      // Запускаем таймер отложенного сброса через 1 секунду
      packetLogTimeout = setTimeout(() => {
        flushPacketLogs();
        packetLogTimeout = null;
      }, 1000);
    }
  } catch (e) {
    console.error('Packet log error', e);
  }
};

// -----
// Механизм heartbeat для обнаружения «мёртвых» соединений.
// При установлении соединения помечаем его как живое.
// При получении pong-ответа от клиента возвращаем метку.
// -----
(ws as any).isAlive = true;
ws.on('pong', () => {
  (ws as any).isAlive = true; // Клиент ответил – соединение активно
});

// -----
// Обработчик входящих сообщений протокола прикладного уровня.

```

```

// Определяет тип сообщения по первому байту (опкоду)
// и выполняет соответствующую операцию.
// -----
ws.on('message', (message: Buffer) => {
  try {
    const type = message[0]; // Опкод – первый байт сообщения
    const size = message.length;

    // =====
    // Опкод 0x00 – AUTH: аутентификация клиента.
    // Формат: [0x00][JWT-токен в байтах UTF-8]
    // =====
    if (type === 0x00) {
      logPacket('in', 'auth', size, message);

      // Извлекаем JWT-токен из байт начиная с позиции 1
      const token = message.subarray(1).toString('utf-8');
      try {
        // Верифицируем токен – проверяем подпись и срок действия
        authenticatedUser = jwt.verify(token, JWT_SECRET);

        // Сохраняем данные пользователя в объекте соединения
        // для отображения в списке активных подключений
        (ws as any).username = authenticatedUser.username;
        (ws as any).connectedAt = Date.now();

        // Отправляем клиенту подтверждение успешной аутентификации
        ws.send(JSON.stringify({ type: 'auth_success' }));
        logPacket('out', 'auth_ack', 0, 'auth_success');
      } catch (err) {
        // Токен недействителен – отправляем ошибку и закрываем соединение
        ws.send(JSON.stringify({ type: 'error', message: 'Invalid
token' }));
        logPacket('out', 'auth_error', 0, 'Invalid token');
        ws.close();
      }
    }

    // =====
    // Опкод 0x01 – FILE_START: начало загрузки файла.
    // Формат: [0x01][fileId: uint32 LE][metadata: JSON UTF-8]
    // =====
  } else if (type === 0x01) {
    if (!authenticatedUser) return ws.close(); // Требуем аутентификацию

    // Читаем 4-байтовый идентификатор сессии передачи (Little Endian)
    const fileId = message.readUInt32LE(1);

    // Извлекаем и парсим JSON-метаданные файла (имя и размер)
    const metadataStr = message.subarray(5).toString('utf-8');
    const metadata = JSON.parse(metadataStr);
  }
}

```

```

currentFileId = fileId;
currentBytesReceived = 0; // Сбрасываем счётчик принятых байт

// Создаём запись о передаче в базе данных со статусом 'uploading'
const info = db.prepare(
    'INSERT INTO transfers (file_id, filename, user_id, total_size,
status) VALUES (?, ?, ?, ?, ?)'
    ).run(fileId.toString(), metadata.filename, authenticatedUser.id,
metadata.size, 'uploading');
dbTransferId = Number(info.lastInsertRowid);

logPacket('in', 'start', size, message);

// Записываем событие начала загрузки в журнал
db.prepare(
    'INSERT INTO logs (user_id, username, action, details) VALUES (?, ?,
?, ?)'
    ).run(authenticatedUser.id, authenticatedUser.username,
        'upload_start', `Started uploading ${metadata.filename}`);

// Отправляем команду Worker Thread открыть файловый поток для записи
fileWorker.postMessage({ type: 'start', fileId, filename:
metadata.filename });

// =====
// Опкод 0x02 – FILE_CHUNK: фрагмент данных файла.
// Формат: [0x02][fileId: uint32 LE][данные фрагмента]
// =====
} else if (type === 0x02) {
    if (!authenticatedUser) return ws.close();

    const fileId = message.readUInt32LE(1); // Идентификатор сессии
    const chunk = message.subarray(5); // Данные фрагмента начиная
с байта 5

    // Обновляем счётчики принятых байт
    currentBytesReceived += chunk.length;
    bytesSinceLastMetric += chunk.length;

    logPacket('in', 'chunk', size, chunk);

    // Сохраняем метрики скорости передачи каждую секунду.
    // Частое обновление (каждый чанк) вызывало бы избыточную нагрузку на
БД.

    const now = Date.now();
    if (now - lastMetricTime > 1000 && dbTransferId) {
        const duration = (now - lastMetricTime) / 1000; // Секунды
        const speed = Math.round(bytesSinceLastMetric / duration); //
Байт/сек

        // Сохраняем точку метрики для графика скорости

```

```

        db.prepare(
            'INSERT INTO transfer_metrics (transfer_id, bytes_transferred,
speed_bps) VALUES (?, ?, ?)'
        ).run(dbTransferId, currentBytesReceived, speed);

        // Обновляем прогресс передачи в записи о передаче
        db.prepare(
            'UPDATE transfers SET transferred_size = ? WHERE id = ?'
        ).run(currentBytesReceived, dbTransferId);

        lastMetricTime = now;
        bytesSinceLastMetric = 0;
    }

    // Передаём фрагмент Worker Thread для записи на диск
    fileWorker.postMessage({ type: 'chunk', fileId, data: chunk });

    // =====
    // Опкод 0x03 – FILE_END: завершение загрузки файла.
    // Формат: [0x03][fileId: uint32 LE]
    // =====
    } else if (type === 0x03) {
        if (!authenticatedUser) return ws.close();

        const fileId = message.readUInt32LE(1);
        logPacket('in', 'end', size, message);

        // Обновляем суммарную статистику переданных байт
        db.prepare(
            'UPDATE stats SET total_bytes = total_bytes + ? WHERE id = 1'
        ).run(currentBytesReceived);

        // Помечаем передачу как завершённую и сохраняем итоговый размер
        if (dbTransferId) {
            db.prepare(
                'UPDATE transfers SET status = ?, end_time = CURRENT_TIMESTAMP,
transferred_size = ? WHERE id = ?'
            ).run('completed', currentBytesReceived, dbTransferId);
        }

        // Записываем событие завершения загрузки в журнал
        db.prepare(
            'INSERT INTO logs (user_id, username, action, details) VALUES (?, ?,
?, ?)'
        ).run(authenticatedUser.id, authenticatedUser.username,
            'upload_end',
            `Finished uploading file ID ${fileId}, total bytes: $
{currentBytesReceived}`);

        // Отправляем Worker Thread команду закрыть файловый поток
        fileWorker.postMessage({ type: 'end', fileId });
    }
}

```

```

// Отправляем клиенту подтверждение успешного сохранения файла
ws.send(JSON.stringify({ type: 'file_success', fileId }));
logPacket('out', 'ack', 0, 'file_success');

// Рассылаем широковещательное уведомление всем подключённым клиентам
// чтобы они обновили список доступных файлов
const updateMsg = JSON.stringify({ type: 'file_list_update' });
wss.clients.forEach((client: any) => {
  if (client.readyState === WebSocket.OPEN) {
    client.send(updateMsg);
  }
});

// =====
// ОPCODE 0x04 – DOWNLOAD_REQUEST: запрос на скачивание файла.
// Формат: [0x04][fileId: uint32 LE]
// =====
} else if (type === 0x04) {
  if (!authenticatedUser) return ws.close();

  // Читаем идентификатор файла в базе данных
  const fileId = message.readUInt32LE(1);
  logPacket('in', 'download_req', size, message);

  // Ищем запись о файле в базе данных
  const transfer = db.prepare(
    'SELECT * FROM transfers WHERE id = ?'
  ).get(fileId) as any;

  // Проверяем наличие записи и физического файла на диске
  if (!transfer || !transfer.file_path || !
fs.existsSync(transfer.file_path)) {
    ws.send(JSON.stringify({ type: 'error', message: 'File not
found' }));
    logPacket('out', 'error', 0, 'File not found');
    return;
  }

  // --- Шаг 1: отправляем DOWNLOAD_START (0x05) с метаданными ---
  // Формат: [0x05][metadata: JSON UTF-8]
  const metadata = JSON.stringify({
    filename: transfer.filename,
    size: transfer.total_size
  });
  const metaBytes = new TextEncoder().encode(metadata);
  const startMsg = new Uint8Array(1 + metaBytes.length);
  startMsg[0] = 0x05; // ОPCODE DOWNLOAD_START
  startMsg.set(metaBytes, 1);
  ws.send(startMsg);
  logPacket('out', 'download_start', startMsg.length, startMsg);

```

```

// --- Шаг 2: потоковая передача файла через DOWNLOAD_CHUNK (0x06) ---
// Читаем файл блоками по 64 КБ для эффективной потоковой передачи
const fileStream = fs.createReadStream(
  transfer.file_path, { highWaterMark: 64 * 1024 }
);

fileStream.on('data', (chunk) => {
  // Формируем сообщение DOWNLOAD_CHUNK:
  // [0x06][данные фрагмента]
  const chunkData = new Uint8Array(chunk as Buffer);
  const chunkMsg = new Uint8Array(1 + chunkData.length);
  chunkMsg[0] = 0x06; // Опкод DOWNLOAD_CHUNK
  chunkMsg.set(chunkData, 1);
  ws.send(chunkMsg);
});

fileStream.on('end', () => {
  // --- Шаг 3: отправляем DOWNLOAD_END (0x07) ---
  // Сигнализирует клиенту что все данные переданы
  const endMsg = new Uint8Array(1);
  endMsg[0] = 0x07; // Опкод DOWNLOAD_END
  ws.send(endMsg);
  logPacket('out', 'download_end', 1, endMsg);

  // Записываем событие скачивания в журнал
  db.prepare(
    'INSERT INTO logs (user_id, username, action, details) VALUES
    (?, ?, ?, ?)'
  ).run(authenticatedUser.id, authenticatedUser.username,
    'download', `Downloaded file ${transfer.filename}`);
});

fileStream.on('error', (err) => {
  // Ошибка чтения файла – сообщаем клиенту
  console.error('File read error', err);
  ws.send(JSON.stringify({ type: 'error', message: 'Error reading
file' }));
});

} catch (err) {
  // Общий обработчик ошибок WebSocket-обработчика
  console.error('[Server] Error in WS message handler:', err);
  ws.send(JSON.stringify({ type: 'error', message: 'Internal server error'
}));
}
});

// -----
// Обработчик закрытия WebSocket-соединения.

```

```

// Записывает событие отключения в журнал если пользователь
// был аутентифицирован.
// -----
ws.on('close', () => {
  if (authenticatedUser) {
    db.prepare(
      'INSERT INTO logs (user_id, username, action, details) VALUES
(? , ? , ? , ?)'
    ).run(authenticatedUser.id, authenticatedUser.username,
      'disconnect', 'User disconnected');
  }
});
});

// -----
// Heartbeat-интервал: каждые 10 секунд проверяет все соединения.
// Если клиент не ответил на предыдущий ping – соединение
// принудительно закрывается через ws.terminate().
// Это освобождает ресурсы занятые «зависшими» соединениями.
// -----
const interval = setInterval(() => {
  wss.clients.forEach(ws: any) => {
    if (ws.isAlive === false) {
      return ws.terminate(); // Соединение не отвечает – принудительно
закрываем
    }
    ws.isAlive = false; // Сбрасываем метку – ожидаем pong-ответ
    ws.ping();          // Отправляем ping
  });
}, 10000); // Интервал проверки: 10 секунд

// При закрытии WebSocket-сервера останавливаем heartbeat-интервал
wss.on('close', () => {
  clearInterval(interval);
});

// -----
// GET /api/admin/connections – список активных подключений.
// Возвращает имя пользователя, IP-адрес и время подключения
// для каждого активного WebSocket-клиента.
// -----
app.get('/api/admin/connections', (req, res) => {
  const connections: any[] = [];
  wss.clients.forEach((client: any) => {
    if (client.readyState === WebSocket.OPEN) {
      connections.push({
        username: client.username || 'Anonymous',
        ip: client._socket?.remoteAddress,
        connectedAt: client.connectedAt
      });
    }
  });
}

```

```

    });
    res.json(connections);
  });
}

// Запускаем сервер
startServer();

// =====

// src/db.ts – Модуль инициализации базы данных SQLite.
// Создаёт или открывает файл базы данных, инициализирует
// схему таблиц и выполняет миграции при обновлении структуры.
// Экспортирует единственный экземпляр подключения к БД.
// =====

import Database from 'better-sqlite3';
import fs from 'fs';
import path from 'path';

// Путь к файлу базы данных в рабочей директории проекта
const dbPath = path.join(process.cwd(), 'data.db');

let db;

// Пытаемся открыть существующую базу данных.
// Если файл повреждён – удаляем его и создаём заново.
try {
  db = new Database(dbPath);
  db.prepare('SELECT 1').get(); // Проверочный запрос на читаемость БД
} catch (err) {
  console.error('Database corrupted or failed to open, recreating...', err);
  try {
    // Удаляем основной файл БД и вспомогательные файлы WAL-режима
    if (fs.existsSync(dbPath)) fs.unlinkSync(dbPath);
    if (fs.existsSync(dbPath + '-shm')) fs.unlinkSync(dbPath + '-shm');
    if (fs.existsSync(dbPath + '-wal')) fs.unlinkSync(dbPath + '-wal');
  } catch (e) {
    console.error('Failed to delete corrupted DB files', e);
  }
  db = new Database(dbPath); // Создаём чистую базу данных
}

// WAL-режим (Write-Ahead Logging) отключён для предотвращения
// возможных конфликтов блокировок в данной среде выполнения.
// В production-окружении рекомендуется включить для производительности.
// db.pragma('journal_mode = WAL');
// db.pragma('busy_timeout = 5000');

// =====
// Инициализация схемы базы данных.

```

```

// Все таблицы создаются с IF NOT EXISTS для идемпотентности –
// повторный запуск сервера не затрёт существующие данные.
// =====
try {
  db.exec(`
    -- Таблица пользователей системы
    CREATE TABLE IF NOT EXISTS users (
      id          INTEGER PRIMARY KEY AUTOINCREMENT,
      username     TEXT UNIQUE NOT NULL,      -- Уникальное имя пользователя
      password_hash TEXT NOT NULL,           -- Хэш пароля (bcrypt)
      role         TEXT NOT NULL DEFAULT 'user' -- Роль: 'user' или 'admin'
    );

    -- Журнал событий сервера
    -- Фиксирует входы, выходы, начало и завершение передач файлов
    CREATE TABLE IF NOT EXISTS logs (
      id          INTEGER PRIMARY KEY AUTOINCREMENT,
      user_id     INTEGER,                      -- Ссылка на пользователя
      username    TEXT,                        -- Имя пользователя (денормализация
для быстрого вывода)
      action      TEXT NOT NULL,              -- Тип события: 'login',
'disconnect', 'upload_start', 'upload_end', 'download'
      details     TEXT,                        -- Текстовое описание события
      timestamp   DATETIME DEFAULT CURRENT_TIMESTAMP,
      FOREIGN KEY(user_id) REFERENCES users(id)
    );
  `);

  // -----
  // Миграция: добавление столбца username в таблицу logs.
  // Выполняется при обновлении схемы на уже развёрнутых БД.
  // -----
  try {
    const columns = db.prepare("PRAGMA table_info(logs)").all() as any[];
    const hasUsername = columns.some(col => col.name === 'username');
    if (!hasUsername) {
      db.prepare('ALTER TABLE logs ADD COLUMN username TEXT').run();
    }
  } catch (e: any) {
    // Ошибка миграции не должна прерывать запуск сервера
  }

  // -----
  // Миграция данных: заполнение столбца username для
  // существующих записей журнала через JOIN с таблицей users.
  // -----
  try {
    db.prepare(`
      UPDATE logs
      SET username = (SELECT username FROM users WHERE users.id = logs.user_id)
      WHERE username IS NULL
    `);
  }
}

```

```

        `).run();
    } catch (e) {
        // Ошибка заполнения данных не прерывает запуск
    }

    db.exec(`
        -- Агрегированная статистика сервера (одна строка с id=1)
        CREATE TABLE IF NOT EXISTS stats (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            total_files   INTEGER DEFAULT 0,    -- Общее число успешно принятых файлов
            total_bytes   INTEGER DEFAULT 0    -- Суммарный объём принятых данных в
байтах
        );

        -- Записи о передачах файлов (загрузка и скачивание)
        CREATE TABLE IF NOT EXISTS transfers (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            file_id       TEXT,                -- Идентификатор сессии передачи от
клиента
            filename      TEXT,                -- Оригинальное имя файла
            user_id       INTEGER,             -- Пользователь, выполнивший передачу
            start_time    DATETIME DEFAULT CURRENT_TIMESTAMP,
            end_time      DATETIME,            -- Время завершения (NULL пока идёт
передача)
            status        TEXT,                -- Статус: 'uploading' или 'completed'
            total_size    INTEGER,             -- Ожидаемый размер файла в байтах
            transferred_size INTEGER DEFAULT 0 -- Фактически принятый размер
        );

        -- Журнал отдельных пакетов протокола для инспектора пакетов
        CREATE TABLE IF NOT EXISTS packets (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            transfer_id   INTEGER,             -- Привязка к записи о передаче
            timestamp     DATETIME DEFAULT CURRENT_TIMESTAMP,
            direction     TEXT,                -- Направление: 'in' (от клиента) или
'out' (к клиенту)
            type          TEXT,                -- Тип пакета: 'auth', 'start', 'chunk',
'end', 'ack' и др.
            size          INTEGER,             -- Размер пакета в байтах
            payload_preview TEXT,             -- Первые байты полезной нагрузки в hex
или text
            FOREIGN KEY(transfer_id) REFERENCES transfers(id)
        );

        -- Метрики скорости передачи для построения графика
        CREATE TABLE IF NOT EXISTS transfer_metrics (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            transfer_id   INTEGER,             -- Привязка к записи о передаче
            timestamp     DATETIME DEFAULT CURRENT_TIMESTAMP,
            bytes_transferred INTEGER,         -- Нарастающий итог переданных байт
            speed_bps     INTEGER,             -- Мгновенная скорость в байтах/сек

```

```

    FOREIGN KEY(transfer_id) REFERENCES transfers(id)
  );

  -- Инициализируем строку статистики если её ещё нет
  INSERT INTO stats (id, total_files, total_bytes)
  SELECT 1, 0, 0
  WHERE NOT EXISTS (SELECT 1 FROM stats WHERE id = 1);
`);

// -----
// Миграция: добавление столбца file_path в таблицу transfers.
// Хранит абсолютный путь к файлу на диске сервера.
// -----
try {
  const columns = db.prepare("PRAGMA table_info(transfers)").all() as any[];
  const hasFilePath = columns.some(col => col.name === 'file_path');
  if (!hasFilePath) {
    db.prepare('ALTER TABLE transfers ADD COLUMN file_path TEXT').run();
  }
} catch (e: any) {
  // Ошибка миграции не прерывает запуск сервера
}

} catch (err) {
  console.error('Failed to initialize database schema:', err);
}

// Экспортируем единственный экземпляр подключения.
// Все модули сервера используют этот же объект.
export default db;

// =====
// src/worker.js – Фоновый поток записи файлов на диск.
// Выполняется в отдельном Worker Thread чтобы не блокировать
// основной цикл событий Node.js при интенсивных операциях
// файлового ввода-вывода во время приёма файлов от клиентов.
//
// Взаимодействие с основным потоком через postMessage:
//   Входящие команды: 'start', 'chunk', 'end'
//   Исходящие ответы: 'ack_start', 'ack_chunk', 'ack_end'
// =====

import { parentPort } from 'worker_threads';
import fs from 'fs';
import path from 'path';

// Директория для сохранения загружаемых файлов
const uploadsDir = path.join(process.cwd(), 'uploads');

// Создаём директорию если она не существует.

```

```

// { recursive: true } предотвращает ошибку при уже существующей директории.
if (!fs.existsSync(uploadsDir)) {
  fs.mkdirSync(uploadsDir, { recursive: true });
}

// Словарь активных потоков записи файлов.
// Ключ – идентификатор сессии передачи (fileId),
// значение – объект WriteStream для записи в файл.
// Хранит одновременно открытые потоки для нескольких параллельных передач.
const activeStreams = new Map();

// =====
// Обработчик сообщений от основного потока сервера.
// Принимает команды управления жизненным циклом файла:
// открытие, запись фрагментов и закрытие потока.
// =====
parentPort?.on('message', (msg) => {
  const { type, fileId, data, filename } = msg;

  if (type === 'start') {
    // -----
    // Команда 'start' – открыть новый файловый поток для записи.
    // Получает: fileId (число), filename (строка).
    // Формирует безопасное имя файла и создаёт WriteStream.
    // Отвечает: 'ack_start' с путём к созданному файлу.
    // -----

    // path.basename удаляет возможные пути из имени файла
    // для предотвращения атак типа path traversal
    // (например, filename = '../../etc/passwd')
    const safeFilename = path.basename(filename);

    // Добавляем временную метку к имени файла чтобы избежать
    // конфликтов имён при загрузке одноимённых файлов
    const filePath = path.join(uploadsDir, `${Date.now()}_${safeFilename}`);

    // Открываем поток записи и сохраняем его в словаре по fileId
    const stream = fs.createWriteStream(filePath);
    activeStreams.set(fileId, stream);

    // Сообщаем основному потоку путь к файлу для сохранения в базе данных
    parentPort.postMessage({ type: 'ack_start', fileId, filePath });
  } else if (type === 'chunk') {
    // -----
    // Команда 'chunk' – записать фрагмент данных в файл.
    // Получает: fileId (число), data (Buffer с данными).
    // Находит поток по fileId и записывает данные на диск.
    // Отвечает: 'ack_chunk' для подтверждения записи.
    // -----
    const stream = activeStreams.get(fileId);

```

```

    if (stream) {
        // Buffer.from гарантирует корректную работу с бинарными данными
        // вне зависимости от типа полученного объекта data
        stream.write(Buffer.from(data));
        parentPort.postMessage({ type: 'ack_chunk', fileId });
    }

} else if (type === 'end') {
    // -----
    // Команда 'end' – завершить запись файла.
    // Получает: fileId (число).
    // Закрывает поток записи и удаляет его из словаря активных потоков.
    // Отвечает: 'ack_end' после закрытия потока.
    // -----
    const stream = activeStreams.get(fileId);
    if (stream) {
        stream.end(); // Сбрасывает буфер и закрывает файл
        activeStreams.delete(fileId); // Освобождаем память
        parentPort.postMessage({ type: 'ack_end', fileId });
    }
}
});

```